



PDF Download
3767734.pdf
16 January 2026
Total Citations: 0
Total Downloads: 881

 Latest updates: <https://dl.acm.org/doi/10.1145/3767734>

RESEARCH-ARTICLE

Topical Analysis of Programming Education Research Literature

LAURI MALMI, Aalto University, Espoo, Uusimaa, Finland

SONSOLES LÓPEZ-PERNAS, University of Eastern Finland, Kuopio, IS, Finland

HUNG NGUYEN, Aalto University, Espoo, Uusimaa, Finland

MIKKO VILLE APIOLA, University of Eastern Finland, Kuopio, IS, Finland

MOHAMMED SAQR, University of Eastern Finland, Kuopio, IS, Finland

Published: 15 November 2025

Online AM: 16 September 2025

Accepted: 28 August 2025

Revised: 25 August 2025

Received: 14 August 2024

[Citation in BibTeX format](#)

Open Access Support provided by:

Aalto University

University of Eastern Finland

Topical Analysis of Programming Education Research Literature

LAURI MALMI, Department of Computer Science, Aalto University, Espoo, Finland

SONSOLES LÓPEZ-PERNAS, School of Computing, University of Eastern Finland, Joensuu, Finland

HUNG NGUYEN, Department of Computer Science, Aalto University, Espoo, Finland

MIKKO APIOLA and MOHAMMED SAQR, School of Computing, University of Eastern Finland, Joensuu, Finland

Programming is taught globally in universities and in a large number of countries, also in schools, often associated with learning computational thinking. It is therefore natural that programming education research is the widest research area within Computing Education Research. Much of this research has taken place and published in conferences and journals associated with the ACM SIGCSE community. However, research related to teaching and learning programming is also published in numerous other venues, which are associated, for example, with the IEEE Computer or IEEE Education societies and other organizations within the fields of engineering education, science education, and educational sciences. Previous general reviews in programming education research have largely focused on the introductory programming level. However, there is much work published addressing more advanced topical areas, too. In this article, the goal is to look at the global situation covering programming education research at different levels. Based on the analysis of over 30,000 papers from the Scopus database addressing computing education and computing education research, we identified 9,865 papers focusing on programming education, published between 1967 and 2023. We describe this filtered dataset and report the results of topic modelling based on titles, abstracts, and keywords in this dataset. We identified 27 topical areas, which form 7 broader themes of research: CS1 and CS2, computing in schools, advanced topics, pedagogy and objectives, evaluation, digital and physical facilities, as well as investigation. We describe work in each of these areas, how research activity has developed in the areas over the decades, how the topics are related to each other, and how work in different areas is more likely to be published in specific conferences and journals/magazines in the field.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Social and professional topics** → **Computing education**;

Additional Key Words and Phrases: computing education, computing education research, literature, research topics, topic modelling, publication venues

Authors' Contact Information: Lauri Malmi (corresponding author), Department of Computer Science, Aalto University, Espoo, Finland; e-mail: lauri.malmi@aalto.fi; Sonsoles López-Pernas, School of Computing, University of Eastern Finland, Joensuu, Finland; e-mail: sonsoles.lopez@uef.fi; Hung Nguyen, Department of Computer Science, Aalto University, Espoo, Finland; e-mail: hung.6.nguyen@aalto.fi; Mikko Apiola, School of Computing, University of Eastern Finland, Joensuu, Finland; e-mail: mikko.apiola@uef.fi; Mohammed Saqr, School of Computing, University of Eastern Finland, Joensuu, Finland; e-mail: saqr@saqr.me.



This work is licensed under Creative Commons Attribution-NonCommercial-ShareAlike International 4.0.

© 2025 Copyright held by the owner/author(s).

ACM 1946-6226/2025/11-ART2

<https://doi.org/10.1145/3767734>

ACM Reference format:

Lauri Malmi, Sonsoles López-Pernas, Hung Nguyen, Mikko Apiola, and Mohammed Saqr. 2025. Topical Analysis of Programming Education Research Literature. *ACM Trans. Comput. Educ.* 26, 1, Article 2 (November 2025), 47 pages.

<https://doi.org/10.1145/3767734>

1 Introduction

Research in programming education has been carried out since the 1960s (e.g., [200]). This work has intensively focused on novice programmers, covering both undergraduate studies in universities and, more recently, K-12 level programming, including computational thinking. For example, the review by Luxton-Reilly et al. [136] identified 1,666 papers on introductory programming published only in the years 2003–2017. However, in addition to the huge work focusing on analyzing introductory programming, there is also considerably research, although in much smaller scale, that has focused on more advanced areas, for example, event-based programming [135], cloud computing [103], computer networks [172], parallel programming [62], computer graphics [210], or software testing [185].

While there are numerous review papers addressing programming education, they have focused the search to specific publication venues, e.g., ACM SIGCSE-associated venues [31, 191] or ACM Digital Library and IEEE Xplore [239], or they have a limited covered time scale (e.g., Valentine [229] (1984–2003), Luxton-Reilly et al. [136] (2003–2017)), or they focus on some specific target area in the field, as in the examples above. None that we are aware of covers the field as a whole, including the wide history. This is understandable due to the scope of research carried out for many decades. However, such an overview would be beneficial for PhD students starting their research in a topic related to programming education, as well as for computing teachers who would like to better understand which areas in programming education have been researched extensively and which have received little attention. Gaining some historical perspective on how research in the area has developed in different subareas would be beneficial for both groups, as well.

In contrast to systematic reviews, which have a focused but limited scope, in this research, we seek to provide such a holistic view on the development of programming education research. We presume this overview to be of interest for scholars and practitioners ranging from newcomers looking for unexplored research niches to practicing teachers and more distinguished academicians interested in gaining a comprehensive view of programming education research.

Another useful perspective for researchers in this area would be getting a broader understanding of venues where programming education papers are actively published, as well as the publication profiles of different journals and conferences in terms of which topics are mostly published in them. While journal web sites and conference call-for-papers naturally provide such guidance, this information can be complemented by a topical analysis of the published papers, thus providing a richer view of the venues. Such information may also support journal editors and conference program chairs when revising web sites presenting the venue profile.

In this article, we present our results of analyzing almost 10,000 papers concerning programming education. The huge volume of research makes manual analysis and categorization quite an infeasible task. Therefore, we apply scientometric analysis methods, which allow us to build a broad view of the field and its subfields. Scientometrics also provides opportunities to identify different trends and relations of subfields, profiles of publication venues, as well as authors. We are not aware of other scientometric analyses of programming education literature.

We seek to build a holistic picture of research in programming education without time or venue constraints to obtain a view of the historical development of the whole programming education

field and its subfields, covering research widely across different publication venues. We therefore investigate research papers indexed in the Scopus database, which covers scientific literature published in a vast number of publication venues (both journals and conferences), much wider than are available at the ACM Digital Library or IEEE Xplore. Moreover, Scopus provides great support for bibliographic and scientometric analysis in terms of including rich metadata of the publications [27]. We seek to identify relevant literature in this vast database and analyze what topical areas of programming education can be identified, in addition to research on introductory programming, as well as where work in different subareas is typically published. Our work was inspired by a recent book, *Past, Present and Future of Computing Education Research* [14], which presented an extensive scientometric analysis of computing education and **Computing Education Research (CER)** literature, also based on Scopus metadata. The chapters in the book, however, analyze the CER field as a whole, while we focus on the literature on programming education. Furthermore, we seek to cover programming education research broadly, not focusing only on the level of introductory programming, as there is much interesting and relevant work related to intermediate- and advanced-level software development.

We discuss the related work more closely in Section 2, followed by presenting our research questions in Section 3. We detail our search and analysis methods in Section 4 and present the results in Section 5. The results and implications are discussed in the Sections 6 and 7.

2 Related Work

Programming education research emerged in the late 1960s and 1970s [91]. Important early works included Papert's studies on children learning programming with Logo and turtle graphics [77, 166]. Researchers also investigated the differences between novices and experts concerning approaches to coding [157] as well as their challenges, covering what type of errors novices made [250] and how they tried to resolve them [157]. From the early years, introductory programming education has expanded and received huge attention in terms of publications. In the first broad review of this area in 2003, Robins et al. [180] identified several areas of research, including the challenges of the programming task itself, mental models involved, novices' capabilities and behavior, differences between novices, as well as teaching and curriculum design. Pears et al. [167], on the other hand, split research into work with curricula, pedagogy, language choice, and research on various types of tools to support learning programming. Luxton-Reilly et al. [136] carried out a systematic review of research in introductory programming, identifying 1,666 papers published in 2003–2017 which focused on four broad areas. The first area focuses on *students*: what and how they learn, their ability, sentiments, i.e., attitudes, engagement and experiences, students at risk, as well as minority groups. The second area covers *teaching*, including theories and models of learning, course structures, teaching techniques and activities, tools supporting teaching and learning, as well as infrastructures and environments supporting teaching. The third area includes research on *curriculum*: what topics are or could/should be included in introductory programming courses, including selecting programming languages and paradigms. Finally, the fourth area covers *assessment*: how students can be assessed, theories/models of assessment, examinations, tools supporting assessment and giving feedback to students, as well as academic integrity and plagiarism.

Another major review of research on novice programming was compiled by Robins [181]. He first discusses the nature of students' learning outcomes, which are often fragile, leading to dropping out. On the other hand, many students perform very well, which may produce bimodal course results. Challenges in the learning outcomes are closely connected to the requirements of learning programming, both declarative/semantic and procedural knowledge, as well as various types of perceived difficulties and errors. Second, he discusses novices and their characteristics, their strategies in programming, as well as mental models and their cognitive load. Evaluating students' progress

when studying programming is a long-term challenge in research. Some major approaches to classify progress or level of outcomes have been adapting Bloom's [119] or Solo taxonomies [38] into a programming context, as well as the neo-Piagetian framework [126]. However, measurement is not enough; we should also understand the underlying factors that influence students' progress or failure, an area with rich empirical work. Finally, Robins also addresses teaching, underlining the reality that there is no joint agreement on what topics introductory programming courses should cover. Therefore, the field has a multitude of papers which present various pedagogical approaches and interventions to support learning (using many different programming languages and different programming paradigms), for example, blended learning [5], computer-supported collaborative learning [198], active learning approaches [36], flipped classroom [96], and pair programming [92, 93]. For more information, see, e.g., the chapter by Falkner and Sheard [76].

Introductory programming courses are often very large in terms of enrolled students. On the other hand, learning programming requires a lot of practical exercises and engagement with learning resources. To address these challenges, educators have developed a large number of different software tools to support teaching and learning [145]. An early review by Valentine [229], covering 444 papers addressing CS1/CS2 in SIGCSE Technical Symposium over the period 1984–2003, identified 99 papers concerning tools. The more recent review by Luxton-Reilly et al. [136] found 268 papers concerning tools for teaching, learning, and assessment of programming. Various categories of tools include, for example, program visualization [203], algorithm visualization [188], automatic assessment and feedback on programming exercises [156, 164], plagiarism detection [6, 163], games for learning programming [197, 223] as well as robots [140] and intelligent tutoring systems for learning programming [55].

While the above reviews covered introductory programming, there are several more specific reviews, which cover various more advanced areas of programming, such as event-driven computing [135], cloud computing [103], computer networks [172], accessibility [28], parallel programming [62] and computer graphics [210], testing [185] and code reviews [82].

In addition to these numerous review papers, the CER literature has also been subject to bibliographical and scientometric analysis. Several papers in a recent book [14] addressed the literature in the field, focusing on authors [18], publication venues [15], and research topics [17]. Similar types of scientometric studies have also addressed computer engineering education [134], digital natives in scientific literature, as well as publications in two important conferences, SIGCSE Technical Symposium and Koli Calling [16, 132].

3 Scientometrics

Scientometric research is concerned with the study of the scientific literature to map the structure, evolution, and impact of scientific knowledge [158, 232]. While the field traditionally focused on citation counts, it has grown to be a robust science that takes advantage of a wide array of statistical and mathematical methods to analyze various aspects of scientific publications, including authorship, publication trends, and the interrelationships among different research entities such as authors, references, institutions, and journals. The insights are used by researchers and policymakers to understand the development of scientific fields and the effectiveness of research or funding. The recent advances in computational data mining have further expanded the repertoire of tools that are used in scientometrics. The use of natural language processing, advanced network analysis, and machine learning allowed for a deeper understanding of a large amount of data with novel aspects beyond the simple citation counts.

While literature reviews have always been the main tools of literature synthesis, their application is limited to smaller datasets and narrow research questions due to the intensive amount of work needed. Scientometrics allows the synthesis of broader research equations and allows the reflection

on the big picture. Of course, there is a tradeoff between scope and specificity, wherein literature reviews are more specific and scientometrics are broader in scope.

In this article, we go beyond the traditional approach of bibliometrics that uses keyword frequency to understand the thematic structure of the field and its evolution. Nevertheless, relying solely on keywords is largely limiting because it overlooks the contextual nuances, the latent relationships between keywords and their variations. In fact, keywords offer a fragmented view because they do not account for synonyms, disciplinary variability, or variations in language usage, leading to potential under-representations of the actual subject matter. This is often compounded by the fact that keywords lack standardization or broad agreement. Oftentimes, researchers do not mention keywords because the topic has become very common, e.g., “machine learning,” and use more specific keywords like “random-forest”; in that case, a better method is needed to capture the full breadth of nuances in the data.

Topic modelling offers a refined approach to textual analysis that involves uncovering latent themes through probabilistic analysis of word co-occurrences and statistical relationships, thereby consolidating variations such as synonyms and differing contextual usages into coherent topics [179]. Furthermore, topic modelling takes advantage of all the textual data (title, abstract, keywords), which provide a rich source for analysis. In doing so, topic modelling provides a more accurate representation of the data. Recent advances in topic modelling facilitate adjusting the technique to allow for a balanced representation of specificity and generality, enabling a more precise and insightful interpretation of the text compared to traditional keyword-based methods [155, 179, 236].

To the knowledge of the authors, there are no prior scientometric analyses of the whole programming education research literature. When exploring the Scopus database through keyword search as well as identifying publication venues focusing solely on computing education (see details in the next section), we identified roughly 10,000 publications addressing programming education. Manual analysis of such a vast dataset was out of the scope of our resources. However, scientometrics provides algorithms for managing large sets of data, as well as visualizing results in a meaningful way. Therefore, the research questions for this work were set as follows:

- RQ1: What are the main research themes of programming education in terms of authors’ keywords and topic modelling?
- RQ2: How has research activity in terms of published papers developed in the topical areas?
- RQ3: What are the topical profiles of the most popular publication venues?
- RQ4: How the topical areas are connected to each other?

4 Methods and Data

4.1 Data Pool

4.1.1 Data Source. Programming education papers are published in numerous conferences, journals, and book chapters. Among the major sources for scientific papers, we selected the Scopus database for our search. This is because Scopus covers the literature from a vast number of venues and has rich metadata concerning authors, institutions, and venues, which are highly useful for obtaining deeper insights [27].

Instead of searching exclusively for papers focusing on programming education, we first collected papers focusing on CER and then later distilled programming education papers from this wider dataset. In this way, we aimed to capture many relevant papers that do not use typical programming-related keywords in their title/abstract/keywords.

To obtain a comprehensive dataset of CER-focused papers, we came up with two strategies:

- *Venue-Based Search.* We compiled a list of the most reputable CER-focused venues and collected papers published in these venues. The complete list of the venues used for this search is given in Appendix A.
- *Keyword-Based Search.* We carefully selected a list of keywords representing the research topics prevalent in the CER field, adopting the basic search query used in previous research [14] and adding several keywords relevant for programming education. We added more wildcards to capture more variations of keyword usage. With this list, we targeted papers whose titles, abstracts, and keywords match the specified topics. The final list of search terms is as follows: “COMPUTING EDUCAT*” OR “COMPUTING STUDENT*” OR “COMPUTING LEARN*” OR “LEARN* COMPUTING” OR “TEACH* COMPUTING” OR “COMPUTING TEACH*” OR “COMPUTING COURSE*” OR “COMPUTING CURRICUL*” OR “INTRODUCTORY COMPUTING” OR “COMPUTER SCIENCE EDUCAT*” OR “COMPUTER SCIENCE STUDENT*” OR “COMPUTER SCIENCE LEARN*” OR “LEARN* COMPUTER SCIENCE” OR “TEACH* COMPUTER SCIENCE” OR “COMPUTER SCIENCE TEACH*” OR “COMPUTER SCIENCE COURSE*” OR “COMPUTER SCIENCE CURRICUL*” OR “INTRODUCTORY COMPUTER SCIENCE” OR “CS EDUCAT*” OR “CS STUDENT*” OR “CS LEARN*” OR “LEARN* CS” OR “TEACH* CS” OR “CS TEACH*” OR “CS COURSE*” OR “CS CURRICUL*” OR “INTRODUCTORY CS” OR “PROGRAMMING EDUCAT*” OR “PROGRAMMING STUDENT*” OR “PROGRAMMING COURSE*” OR “PROGRAMMING CURRICUL*” OR “INTRODUCTORY PROGRAMMING.”

The results from the two searches were then combined into a single dataset with 30,415 results.

4.2 Data Cleaning

Overall, the raw data needed significant cleaning to make it usable for further analysis. The main issue was that many of the same elements were written in completely different formats. Moreover, because our search query was aimed toward obtaining a comprehensive list of papers, the dataset also included many irrelevant papers. The cleaning process took place in several phases, as described in Figure 1.¹

4.2.1 Preprocessing. In the initial step, we employed Biblioshiny, an interactive interface provided by the R library bibliometrix [23], to transform raw data sourced from Scopus into a more systematic and reproducible format. The outcome of the process is the same set of data, but with minimally renamed columns and new features based on existing ones to work with.

Following our list of CER-focused venues, we then tracked each venue by each year to check for potential missing/invalid data and addressed them where possible. For instance, the host year of the *37th ACM SIGCSE conference (2006)* was wrongly labelled as 2007 by Scopus, or in some cases *ITiCSE* papers were marked as from *SIGCSE* in their sources.

4.2.2 Unifying Formatting Inconsistencies. In the next step, our goal was to address the inconsistencies in the name formatting of many elements, such as venues and keywords. Following the steps of [133], we utilized OpenRefine—an open source data-cleaning software developed by Google—to recognize and merge values with the same meanings effectively. To describe the problems and our approaches in detail:

- *Venues’ Names.* Many venues’ names have varied substantially over the years and should be unified. For example, the “ITiCSE” conference was initially “Integrating Technology into

¹As this article focuses on paper contents and venues only, and the results of authors and affiliations will be reported elsewhere, we report the cleaning process only when it concerns relevant aspects of this article.

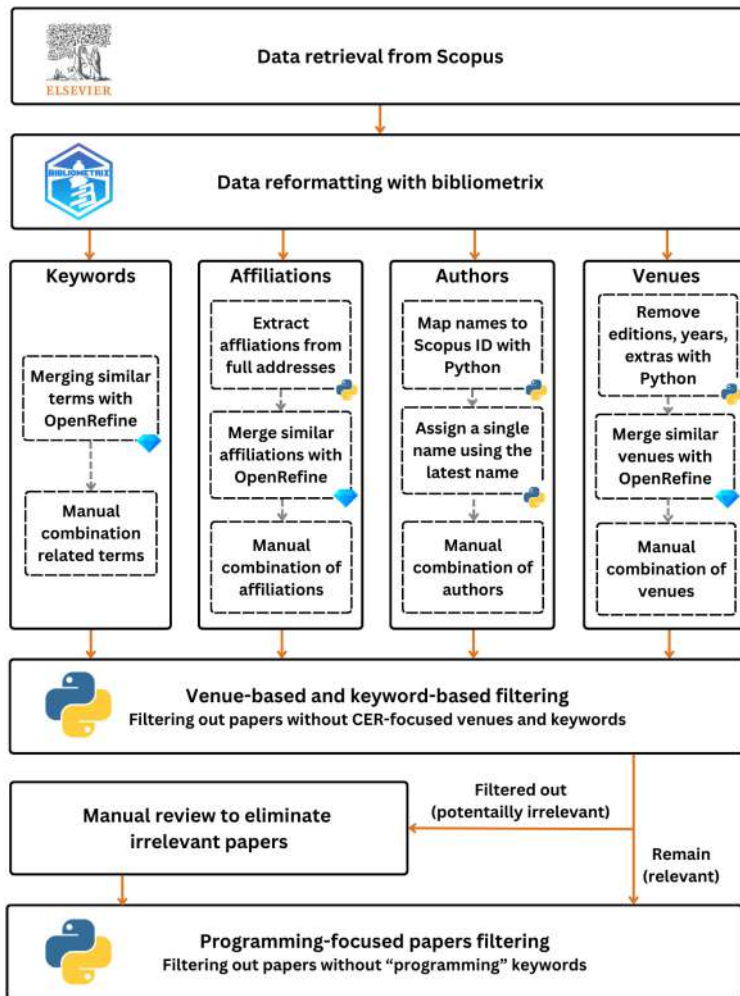


Fig. 1. Data cleaning process.

Computer Science Education” and later changed to “Innovation and Technology in Computer Science Education.” Proceedings’ titles vary from year to year, sometimes using different edition numbering formats, “14th”–“fourteenth”; some may also include additional details such as conference year. Moreover, many proceedings have extra words in their titles like “Annual”, “Joint”, or “ACM SIGCSE.” We removed such redundant terms to make the names more similar, making them easier to identify and merge using OpenRefine. To ensure that all the more important (CER-focused) venues are correctly handled, we manually reviewed and combined each venue as needed.

- **Authors’ names** can be written in many ways and change in time. This issue was addressed by matching the unique Scopus Identifier (Scopus ID) assigned to each author to unite their name(s) to the latest version in the database.
- **Keywords.** The keywords, which represent the main theme being discussed in each paper, are purely determined by the authors themselves, resulting in inconsistencies between the keywords of each paper. The main differences found in the data were addressed as follows:

- *Inconsistencies in Spelling*. Many words are written differently, e.g., “visualization” vs. “visualisation,” “online learning” vs. “online-learning,” “university” vs. “universities.” We utilized the Phonetic algorithm to detect words with the same pronunciation, and the nearest neighbors algorithm to find words with close spellings.
- *Abbreviation of Keywords*. Some words are written as the abbreviated version of another: “OOP” vs. “OBJECT-ORIENTED PROGRAMMING,” “CS” vs. “COMPUTER SCIENCE.” We decided to manually clean them by utilizing a spreadsheet to search for such keywords. However, only words that had frequencies of five or more were considered, as it would have been impractical to consider every single keyword, including the infrequent ones. The same approach was used to unify keywords with the same meaning that can be written into different versions as synonyms: “FRESHMAN” vs. “FIRST YEAR STUDENT,” “CS1/CS2” vs. “INTRODUCTORY PROGRAMMING.”

4.2.3 Eliminating Irrelevant Papers. We checked for possible duplicate papers, which could be, for example, reprints of the best papers. Moreover, formerly proceedings of the SIGCSE technical symposium and ITiCSE were also published in the SIGCSE Bulletin, creating a large number of duplicates. Duplicates were removed based on title comparison.

As mentioned, to capture a comprehensive list of CER-focused papers, it was necessary to aim for a more open list of topics, which would also introduce many irrelevant papers. In the final step, our goal was to eliminate these irrelevant papers from the dataset.

We performed manual screening to ensure that no relevant papers were mistakenly removed from the data. Our methodology was based on the assumption that the content of a paper could be represented by its title and abstract. This allowed us to determine a paper’s basic content without having to read through the entire paper.

Nevertheless, it would have been impractical to manually screen all 30,415 papers in the data. Therefore, we employed an inclusion/exclusion strategy to omit well-defined CER papers (that would be moved immediately to the final cleaned dataset) from the scope of this cleaning process, allowing us to focus on the most likely instances of irrelevant data:

- We had access to the dataset used in, which included 17,678 computing education papers till 2021, which were therefore considered relevant. We merged this dataset with our own new dataset, reaching till 2023, and eliminated overlapping papers.
- We eliminated complete proceedings from the dataset, as we were interested in the individual papers only.
- Venue-based elimination: Undoubtedly, papers from CER-focused venues could be considered relevant. In addition to our initial list of such venues, we skimmed through all venue names from a sorted list and identified several other CER-focused venues (see Appendix A). All papers in these venues were included in the final dataset and excluded completely from the irrelevant papers cleaning process.
- Keyword-based elimination: In the remaining dataset, there were numerous papers with keywords belonging specifically to the CER field, such as “TEACHING CS” or “LEARNING PROGRAMMING.” We formed another list of keywords to identify papers with titles and abstracts matching this set of keywords to identify papers that should be included in the final dataset and excluded from the later cleaning process.

After the above screening process, there were only 3,712 papers left in the data pool that needed further manual cleaning. For this process, we had two researchers individually screen the papers to identify cases for inclusion or exclusion. In most cases, the title alone was sufficient to conclude that the paper deals with computing education and could be included. If the title was not clear

enough, the abstract was read to make a clear decision. However, there were many borderline cases (about 2–3% of data in this phase), which were noted by the researchers and which were jointly discussed until a consensus for inclusion/exclusion for each of them was reached. The following principles were formed in these discussions:

- If the paper addresses teaching computing to non-CS students such as CS minors, or students from other disciplines, e.g. programming for engineering or biology students, it will be considered CER-relevant.
- If the paper addresses a setting where some computing course is targeted among some non-computing courses to analyze some broader questions such as students' dropout from a set of **Massive Open Online Courses (MOOCs)**, it will not be considered CER-relevant.
- If the paper addresses a setting where specifically computer science students were learning some non-computing topic which is building their general competences, such as discrete mathematics, teamwork, or academic writing, it will be considered CER-relevant. On the other hand, if such a learning setting includes students from several disciplines and does not focus on computing students, it was not considered CER-relevant. For example, there were many papers addressing a population including computing and engineering students without considering these separately.
- We interpreted aspects related to teaching and learning computing broadly, including learning resources, educational tools, student recruitment, retention, background, motivational aspects, attitudes, and so on.
- Together we eliminated manually 1,489 papers.

Finally, to obtain the programming-relevant dataset, we filtered all papers with the term “programming” in the title, abstract, or keywords, resulting in the final data pool with 9,865 papers.²

Note about Paper Type. It was not possible to identify whether a paper was a research paper, an experience report, or an opinion piece, as this would have required reading the whole paper. We accepted all of them in the dataset. We also decided to include poster abstracts, as our purpose was to map existing reported research, and posters are frequently used to report early phases of research. We excluded panel descriptions if this was clear from the article type metadata. However, not all cases had such metadata information. We acknowledge that there remained a small number in the data; however, panels frequently address relevant topics, research, and development of education, and therefore, their descriptions actually support building better topic modelling.

We implemented extensive quality control measures in our study. We manually selected every paper that did not originate from a dedicated venue—a step not commonly taken in scientometric studies. We constructed our query based on previous literature and refined it through discussions among five researchers. To ensure that the query captured all relevant papers, we identified the top 10 computing education researchers who published on programming and recorded their publication counts. We then executed a very permissive query designed to capture all relevant articles, even if it initially returned many irrelevant ones. We compared the results with the known publication counts and examined any discrepancies to understand why our keywords might have missed certain articles. We repeated this process until we confirmed that our query effectively captured all articles by the top authors. Finally, we scrutinized the complete dataset for inclusion using systematic review protocols.

²We acknowledge that the term “programming” could appear in education-related papers focusing on some CS topic, which is not about programming. Manual browsing of hundreds of papers in the cleaned dataset has not revealed such cases, indicating that such false positives are rare.

4.3 Data Analysis

After cleaning the data, we prepared the data for topic modelling based on the available metadata for each paper, as follows: We used the R library UDpipe [206] to preprocess the articles' titles, author keywords, and abstracts. Specifically, we conducted tokenization, **Part-of-Speech (POS)** tagging, and lemmatization. Tokenization involves segmenting the text into smaller units (typically words) for easier analysis. POS tagging is the categorization of each token according to its grammatical role (e.g., pronoun, verb, adjective, noun). Finally, lemmatization means reducing each word to its base or dictionary form (e.g., converting plural nouns to singular, verb forms to their infinitive form). This preprocessing approach is preferable over traditional methods like stemming, which have proven inadequate for structured topic modelling [186]. Next, we applied structured topic modelling with the goal of identifying distinct research topics. Structured topic modelling is a clustering technique that can be used to detect themes in a text corpus [177] and is widely used to identify research topics using article metadata [17]. We employed the R library *stm* [178], which combines the Latent Dirichlet Allocation algorithm with a variational Expectation-Maximization algorithm. Since the optimal number of topics is not known beforehand, we estimated several models with different topic numbers (ranging from 5 to 40). We determined the best model using a combination of quantitative fit indices (semantic coherence and exclusivity) and qualitative criteria based on the judgment of the authors to maximize thematic coverage and interpretability while minimizing the overlap between topics. When qualitatively comparing the different models, we looked at the 10 most frequent terms in each topical cluster, seeking to identify clusters with a coherent set of terms. We observed that models with less than 20 or more than 30 clusters had many clusters with low coherence, despite that some topical clusters were present in most models. A closer investigation of models between 20 and 30 clusters was then carried out, and a consensus was reached that 27 clusters had the best overall coherence. These topical clusters were then labeled with corresponding initial names (topics). Furthermore, we skimmed through a significant part of the titles in each cluster, as well as the presence of relevant keywords in the abstracts, to get a better understanding of the content and used this information to define the final labels such that they appropriately characterized most papers in the cluster.

As the total number of distinct topics was so large, we concluded that it would be reasonable for further analysis and reporting to classify them into broader categories, which we call *themes*. This was carried with joint discussion with the authors until consensus was reached.

We acknowledge that for this kind of analysis, where data are human-authored text, there is an inherent limitation that a single article could be categorized into more than one category. This holds for both interpretive manual categorization and topic modelling. In topic modelling, the factor *gamma coefficient* reports how strongly an article is associated with each topic, and we categorize papers based on their highest gamma values. To map the temporal evolution of the identified topics, we plotted each topic and category based on their frequency over time. For each article, we selected the most representative topic (the one with the highest gamma value) and plotted the number of articles per topic (and category) per year.

To explore how topics appear in combination with one another, we constructed a network where nodes represent topics, and edges signify co-occurrence within the same article. We considered the top 5 topics (based on gamma) per article, as these were deemed to reflect the most salient themes (consistent with the common practice of selecting five keywords per article). Topic assignment probabilities (gamma, representing the degree of association between a document and a topic) were then utilized as edge weights to quantify connection strength. Network construction considered the co-occurrence of these top topics within articles. Each topic was treated as a node, and the edge weight between topics was determined by aggregating gamma values from articles where both

topics were present. We plotted the network with the Gephi network visualization software [30]. Subsequently, to identify frequently co-occurring topics, we employed community detection via the Louvain modularity algorithm, which delineates groups of densely interconnected nodes. The algorithm optimizes modularity, a metric that evaluates the density of edges within communities relative to the expected density in a random network. It initiates with each node in its own community and iteratively reassigns nodes to maximize modularity. Upon reaching maximal modularity, the algorithm aggregates communities and reiterates the process. This method reveals clusters of topics exhibiting strong internal connections, thereby highlighting coherent thematic groupings within the network structure [61].

Last, to visualize the distribution of topics across venues (journals and conferences), we created a heatmap with topics on the y -axis and venues on the x -axis. The color intensity in each cell of the heatmap is proportional to the mean gamma value for the topic, calculated by averaging the gammas of all the articles published in the venue.

5 Results

5.1 Data Description

The total number of documents in the data was 9,865, the majority of which were published in conference proceedings, 7,045 (71.4%), reflecting the publishing tradition of computer science (Table 1). The oldest document in the data dates back almost 60 years ago [200], which discussed efforts students have to make to learn programming. The average citations per document—within Scopus³—were 9.4, a rate higher than computing education in general, which is 7.8 [19], and also lower than 14.38 of education technology in general [230]. The data contained 16,033 authors who appeared 28,452 times on different papers with an average of 2.88 authors per document. Compared to computing education or education technology in general, programming education has a higher number of authors collaborating on the same paper [230]. Most interestingly, the average age of a document is 11.4 years, which is relatively short compared to other established fields [230], although higher than more recent research areas like computational thinking [184]. The average growth rate (the arithmetic mean of the changes in the number of documents from one year to the next) is 12.3%, pointing to a vast rate of growth of the research corpus.

Figure 2 presents a summary of the data in graphical form. There is a huge increase in papers (Figure 2(a)) from roughly 2000 to 2020, which is likely explained by the growing number of computing teachers and computing education researchers. For example, during this period, computing or computer science was increasingly included in school curricula, and naturally, this area gained a lot of interest in educational research. The significant extension of educational research [224] more generally matches well with our findings in computing and programming education.

After 2020, the number of published papers per year in our dataset seems to decrease. While we do not have a comprehensive explanation for this observed decrease, there are two likely reasons for it. First, the COVID-19 pandemic caused a disruption in the academic world, as teachers had to use very much time transferring education from the classroom to a fully online format, which likely reduced their research capacity. Second, concerning the last year, the Scopus database lags somewhat behind the actual publications, as it takes time to harvest papers from numerous publication venues into the database. Thus, the data concerning 2023 were not complete when the current analysis was completed in spring 2024.

The other figures depict a huge skew in the data. The number of different authors is very large; however, a large share of them have authored only one paper in the dataset (Figure 2(b)). Moreover,

³For clarity, we use the term *citation* when another paper within Scopus has a reference to the current paper. We use the term *reference* for the references within the target paper itself.

Table 1. Descriptive Statistics about the Bibliometric Dataset

MAIN INFORMATION ABOUT DATA	
Timespan	1967–2023
Sources (Journals, Books, etc.)	1,954
Documents	9,865
Annual growth rate %	12.3%
Document average age	11.4
Average citations per doc in Scopus	9.43
DOCUMENT TYPES	
Article	2,597
Book	31
Book chapter	108
Conference paper	7,045
Review	84
AUTHORS	
Authors	16,033
Author appearances	28,452
Single-authored docs	2,106
Co-authors per doc	2.88
Documents per author	0.615
International co-authorships %	10.31%

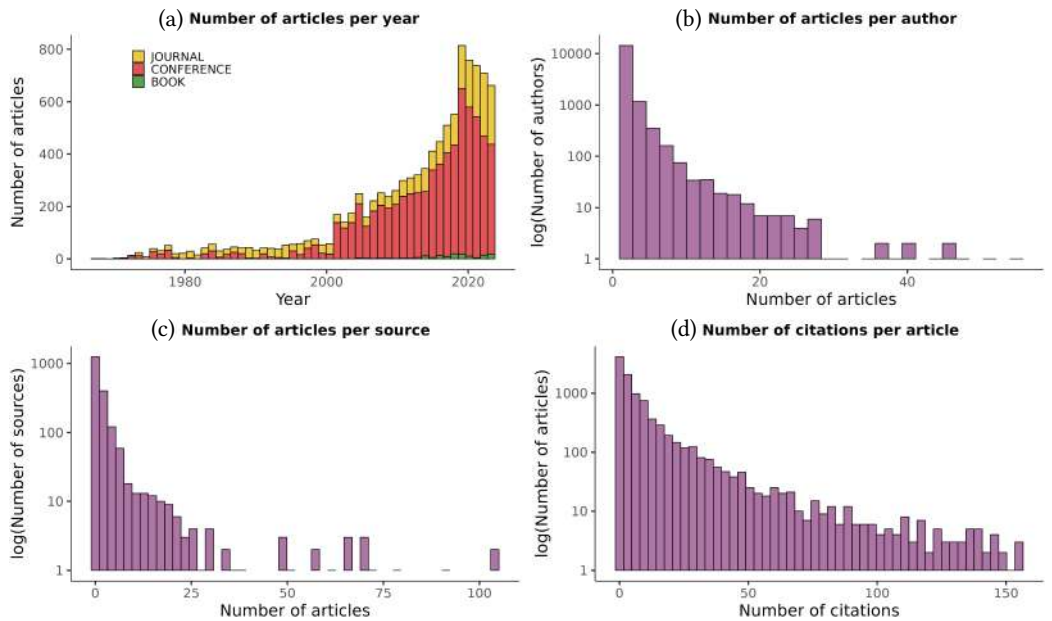


Fig. 2. Descriptive statistics of the analyzed dataset. Note: y-axes in (b), (c), and (d) have logarithmic scales. For (c) and (d), outliers (top 0.5% values) have been removed to avoid skewing the graph beyond readability.

programming education papers are published in a vast number of publication venues (journals, conferences, workshops, books). However, almost all of these venues publish very few such papers

(Figure 2(c)), which very likely explains that a very large share of papers have none or very few citations in the data (Figure 2(d)). The largest venue, ACM SIGCSE Technical Symposium, has 1,259 published papers since 1970 in our dataset, thus covering 12.8% of the total data pool.

5.2 Topic Modelling

We carried out structured topic modelling of the articles' metadata (titles, author keywords, and abstracts), concluding that the most fitting model was that with 27 topical areas. Due to the large number of topics, we combined them under 7 broader themes to ease reporting, which are presented below. The themes, topics, and most common terms of each topic are presented in Table 2. As noted above in Section 4.3, a single paper could be categorized in more than one topical area (we used the gamma value to decide which one). In multidisciplinary research, this can be expected, causing some inherent overlap of categories and themes. However, when looking at the categories as a whole, they have characteristics that form a meaningful whole, as will be discussed below. For example, many topics address work carried out in CS1 contexts, but the points of view in the presented research vary significantly, e.g., student performance, psychological factors involved, pedagogical practices, or supporting learning resources. For each topic, we explain its "big picture" and provide several example papers. However, due to the huge scope of research, it is not possible to provide a comprehensive summary of work in each area. We often therefore refer to existing topical review papers.

5.2.1 CS1 and CS2. The first and second programming courses, very often denoted as CS1 and CS2, form the context of a large part of programming education research. This research covers multiple subareas. The theme *CS1 and CS2* covers *Learning resources for coding* (in this context), *Novice coding process and errors* which students face, and *Programming languages* which focuses largely on choice or comparison of the first programming languages. However, the other themes also include much research on introductory programming from other points of view, such as programming tasks and psychological or other factors that influence students' performance or behavior in introductory courses. As a large number of papers in the other categories/themes address many other curricular contexts in addition to CS1/CS2, we discuss them separately within the other themes. Here, we begin with a brief historical view of the area.

Early papers in the 1970s and 1980s focused much on presenting various implementations of introductory courses and how some topics could be instructed in specific programming languages, e.g. [121, 176]. Thereafter, research has gradually focused on a variety of threads. Some examples include addressing learning specific language constructs, e.g., loops [43], or programming techniques, such as structured programming [81], use of recursion [95], using formal logic for proving program correctness [215], or program quality factors (programming style and readability) [233]. More recent threads are manifold, as research in the field has rapidly extended during the last 20–25 years. In their review, Luxton-Reilly et al. [136] found that the most popular areas include first supporting and evaluating students' code writing, reading, tracing, as well as debugging skills [82, 151]. Moreover, as learning environments and integrated development environments can be used to collect a lot of information on students' actions, these data can be analyzed using techniques of learning analytics to track and interpret their general behavior [94, 105]. Students' abilities [209], as well as their attitudes and engagement, have also been subject to active research [4, 142]. Second, a natural wide area of research has been various methods and approaches to teach programming [76]. In parallel with this, introductory programming education has inspired the development of numerous different software tools to support teaching and learning, such as tools for automatic assessment [164], program visualization [203], games [197], and IDE support [145]. Third, research about curricula, i.e., what should be included in programming education, as well as what programming languages

Table 2. Themes and Keywords in Topic Modelling of Programming Education Research

Theme	Count	Topic	Count	Terms
Advanced topics	2,085	AI, ML, modelling	207	model, machine, modelling, intelligence, artificial, mental, recursion, base, network, AI
		Advanced programming	311	parallel, architecture, distribute, concurrent, comput, programming, processing, memory, computing, level
		Logic and formal presentation	257	logic, formal, proof, reasoning, theorem, mathematics, deductive, propositional, predicate, inference
		Mobile programming	179	project, mobile, application, laboratory, app, service, base, lab, device, network
		OO programming	427	concept, object, oriented, design, programming, class, understanding, paradigm, concepts, teach
		Software engineering	400	software, engineering, development, design, process, security, quality, develop, experience, undergraduate
		Web programming	304	java, web, care, class, C++, programme, application, textbook, library, IDE
CS1 and CS2	895	Learning resources for coding	294	program, write, comprehension, debugging, variable, statement, construct, programming, debugger, execution
		Novice coding process and errors	373	code, novice, programmer, error, coding, programming, type, programme, misconception, analysis
		Programming languages	228	language, programming, python, syntax, pascal, english, natural, feature, transition, semantic
Computing in schools	1,282	Computational thinking	657	school, teacher, computational, education, thinking, primary, development, CT, design, child
		K-12 programming environments	350	environment, programming, base, block, visual, user, scratch, programme, text, interface
		Robots and tangible computing	275	robot, robotic, physical, simulation, hardware, control, technology, platform, music, educational

(Continued)

Table 2. Continued

Theme	Count	Topic	Count	Terms
Digital and physical facilities	1,335	Classroom and online activities	423	online, classroom, class, peer, activity, lecture, active, video, engagement, instruction
		Games and gamification	461	game, design, educational, virtual, gamification, education, play, base, programming, digital
		Learning systems	345	system, algorithm, visualization, management, tutor, intelligent, base, tutoring, e-learning, propose
		Supporting tools	106	tool, support, programming, provide, educational, interactive, develop, challenge, context, educator
Evaluation	1,185	Assessment	640	assessment, assignment, feedback, test, testing, programming, automatic, provide, automate, solution
		Plagiarism and academic integrity	98	education, source, plagiarism, detection, similarity, information, comput, propose, academic, ICT
		Student performance	447	question, performance, exam, analysis, behavior, programming, time, answer, measure, score
Investigation	1,028	Psychological factors	554	programming, programme, effect, factor, motivation, performance, difference, student, impact, attitude
		Research on CS1	474	introductory, experience, programming, CS1, survey, rate, success, report, prior, semester
Pedagogy and objectives	2,055	Cognitive skills	393	skill, knowledge, solve, strategy, cognitive, process, theory, programming, ability, understanding
		Programming tasks	63	practice, exercise, programming, task, solve, line, puzzle, competition, require, short
		STEM and curriculum	668	science, computing, curriculum, comput, technology, include, program, college, university, CS
		Teaching	754	programme, programming, teaching, approach, teach, base, program, improve, subject, university
		Teamwork	177	pair, collaborative, programming, team, programme, collaboration, communication, benefit

Counts indicate papers with highest gamma value in the topic.

and paradigms should be used, has been intensive. Major outcomes of curricula development work have been reported by ACM and IEEE CS in their curricula recommendations providing also examples of different types of model implementations for introductory programming [1, 24, 25, 226]. Finally, various aspects of assessment have also received considerable attention [168]. All these areas are visible in our topic modelling, as well.

The category *Novice coding process and errors* covers papers which deal with the programming process, mainly in introductory courses, addressing coding, compilation, testing, and debugging. Another aspect heavily addressed in the papers concerns students' misconceptions and errors that students make. Category *Learning resources for coding* includes a mix of papers focusing on programming learning resources (assignments, tools) and various learning activities mainly in CS1/CS2 contexts, including program comprehension. See some examples in the previous paragraph.

Programming languages taught in computer science education have been shaped by technology advancements and industry trends. The dataset includes numerous papers that discuss the design and implementation of a programming course in a specific language or discuss the language choice, especially for introductory courses. The analysis often focuses on which programming concepts or paradigms are supported in the language.

Thus, in the early articles of our dataset, we see the dominance of assembly language [231], Pascal [222], ADA [221], or LISP [162]. Then, in the early 2000s, we see a shift to Java [70], Visual Basic [101], C# [84], or Ruby [192]. After 2010, there is a huge rise in Python [193] due to the increasing role of data analysis and machine learning in computer science degrees, as well as JavaScript [10] to give an answer to the increasing demand for web development skills. In the early years, imperative/procedural paradigm was dominant, but the functional paradigm with LISP, Scheme, Standard ML, or Haskell has been presented as models from the 1980s onwards (e.g., [49, 52]). **Object-Oriented Programming (OOP)** became highly popular with the emergence of Java around the millennium. Modern programming languages (e.g., Python, Scala, and JavaScript) stand out for their multiparadigmatic nature [42]—procedural, OO, and functional—, which allows students to become acquainted with different programming paradigms without the need to learn multiple languages, which can be overwhelming for novices [66].

5.2.2 Pedagogy and Objectives. Learning programming not only involves learning new concepts and constructs, but rather a new skill that employs many different subskills, such as problem analysis, problem solving, program design, in addition to coding, testing, tracing, and debugging skills discussed above. Therefore, much research has focused on how to support students in learning *Cognitive skills*. Several large-scale international studies have explored students' skills in solving simple CS1 level tasks [152, 228], tracing skills [127], and program design skills [219] identifying serious challenges in meeting learning goals. Numerous papers have presented various types of *teaching* interventions to support building these skills [48, 76, 82, 96, 131, 151, 183, 185, 189, 198, 239]. An example of a more comprehensive development building a theoretical framework is the work by Xie et al. [248] who presented a theory how students gradually learn to read and understand code elements and templates, as well as learn to write them, thus giving a framework for designing how programming exercises could be designed for a whole course.

Designing *Programming tasks* which support learning on an appropriate level when considering students' current progress has been challenging. The literature includes many papers that present various exercises and problems on different levels of programming courses. Many such tasks are algorithmic problems, which are frequently used in programming contests. [78].

Bloom's taxonomy [119] provides one framework for evaluating the task level. However, research has indicated that there are serious challenges in classifying assignments [195] and therefore more elaborate classifications have been proposed [154].

One of the major challenges that students struggle with is programming language syntax [205]. Therefore, block-based languages, such as Scratch, Snap, and Blockly, have become very popular, especially in K-12 level programming education. Visual appearance of the language constructs prevents typical syntax errors compared with text-based languages and allows students to focus on the program logic. Block-based languages are, however, rarely used in a professional context, and the transition to text-based languages is not straightforward. Thus, research has addressed how the pitfalls in this transition could be avoided [225, 242]. One approach seeking to circumvent the challenges is Parsons problems or programming puzzles [74]. They seek to overcome the syntax challenges by presenting the program code on a statement-based level (or wider components), thus presenting the task so that students can focus on algorithm design by ordering given statements to form a working solution.

Introductory programming courses typically request students to solve programming problems alone, with the aim of supporting enough personal training of relevant skills. However, working in pairs and *Team working* has received a lot of attention in research. Pair programming [48, 227] has also been widely used in introductory courses with generally positive results. In the later phase of studies, larger programming projects, often associated with learning software engineering, have an important role, and in these contexts, learning teamworking, as well as exploring students' collaboration and communication skills, are relevant research topics [53, 71]. However, teamwork has also been researched in CS1 level courses, too [100].

Finally, planning *Curriculum* requirements is a central part of designing and implementing programs that focus on different areas of computing. As mentioned above, ACM and IEEE CS have jointly designed recommendations for programs in computer science, software engineering, computer engineering, information systems, information technology, cybersecurity, and data science [1]. In each of these, learning programming is one important objective. Literature covers many papers presenting how curriculum requirements have been designed to match the needs for different institutes, e.g., liberal arts colleges [86, 99]. Another large area concerns broadening participation, i.e., designing various educational activities, such as summer schools, summer camps, boot camps, etc., which are targeted to specific groups of students, such as high school teachers [40], primary or middle school children [208], girls [170], or new groups of students [110, 174]. Computing and programming are also highly relevant in *STEM* (science, technology, engineering, mathematics) programs.

5.2.3 Computing in Schools. Computational Thinking (CT) is a central area of CER, with a strong connection to programming education. While the roots of CT are in the early works of, e.g., Knuth, Perlis, and Dijkstra in the 1950s [217], and in the 1980s groundbreaking contribution of Papert's *Mindstorms* [166], the concept of CT was brought to popular attention in 2006 by Wing [245], who proposed that the increased importance of computational processes makes learning CT beneficial for everyone. This marked a starting point for a rapid acceleration in CT initiatives and research; the number of CT publications reached some 430 articles in 2019, with an annual percentage growth rate of 61.2% [184], reflected also in an upward trend in top keywords [19]. Alternatively, although less popular labels for CT include computational making, and computational participation [108, 109].

According to a common definition, CT refers to the "thought processes involved in formulating problems so their solutions can be represented as computational steps and algorithms" [2, 3]. The definitions of CT have evolved [63, 217], where some align CT more with programming skills [137], while others have emphasized, e.g., aspects related to thought processes [63, 217]. A common breakdown of CT skills has been that of *decomposition*, *abstraction*, *algorithmic thinking*, *iterations*, *debugging*, and *generalization* [196], while CT is also commonly seen to be located in the intersection

of *mathematical thinking*, *engineering thinking*, and *scientific thinking* [246]. In recent years, a trend of CT2.0 suggests adding understanding of how modern systems based on machine learning and AI work and are designed [64, 216]. Some have suggested adding design skills to reflect on the potential impact of technologies to users and societies [20, 57].

Among the diverse number of concrete examples, common approaches include block-based programming [75], educational robotics [194], makerspaces for construction of tangible computational artefacts, informal hacker events [89], and others [90]. Interdisciplinary pedagogies include combining CT with, e.g., music education [33], while pedagogies of open-ended problem solving are also popular [59], examples including, e.g., the well-established Computer Science Unplugged kit [34]. While CT is mostly associated with K-12 context, CT has also been applied in higher education, with examples including game-based or robot programming environments in interdisciplinary STEM contexts [57]. It has been argued that “*the beauty of computing is that it is engineering, science and art, a subject without clear borders*,” while also stressing abilities to think “outside” the domain of computer science and connect CT to addressing societal challenges [57]. In recent years, initiatives have invested more focus on how machine learning techniques work and are designed [235]. One stream of research is that of developing metrics and measurements for assessing CT skills and their development [51, 244].

K-12 Programming Environments. Within the evolution of research on programming environments [113], in addition to tangible computing, two major branches in the K-12 context have been those of text-based and block-based environments [58]. Block-based visual programming has become a major stream of research and has challenged the educational approach of text-based programming [58]. Common block-based approaches include Scratch [146, 147] and Alice, which enables programming via story-based animations [114]. While comparative research has shown higher learning gains and elevated interest toward computing courses resulting from block-based learning compared to text-based learning [241, 242], it has been argued that the amount of comparative research in this area is still remarkably low [58]. Approaches of text-based programming include that of PRIMM (Predict-Run-Investigate-Modify-Make) [187] and are reviewed in more detail in [58]. It has been shown that much of the research on block-based environments has been done at the primary school level and that many environments are also used for teaching CT [58]. Research of K-12 programming environments has strong connections with CT [58].

Robots and tangible computing is one major branch in K-12 CER [58]. Its roots are in the LOGO programming language, which was originally designed based on Piaget’s constructivism and Marvin Minsky’s ideas of AI, and which later became a symbol for pedagogical change and connecting of computing education with language, music, graphics, animation, storytelling, turtle geometry, robotics, and physical devices [202]. A variety of pedagogies have followed, often including a tangible and hands-on element, with a connection to another discipline such as music [33], visual art and textiles [174], or science and simulations [21]. Concrete applications of robots and tangible computing in CER are many, including teaching introductory programming and embedded systems by using a hands-on approach with Lego®Mindstorms, teaching Internet-of-Things based on experimentation pedagogies and active learning [118], using Lego®Mindstorms in an invention-based pedagogical scenario [12, 13], teaching making of e-textiles with a project-based approach [107], teaching computational thinking via simulation in a science context [21], and a proposed curriculum for teaching programming by using computational textiles [174]. The tangible nature of, e.g., seeing the immediate result of a program directly in the physical world, has been found to be a motivating factor for learners, along with a setting that promotes free play and exploration that is often common in childhood [12]. The research track of robots and tangible computing is strongly connected to programming education and computational thinking [58].

5.2.4 Evaluation. *Assessment and Students' performance* have been central topics in programming education. Researchers have investigated the motivational and psychological factors that are associated with students' performance in programming [35]. Long before the field of learning analytics and educational data mining emerged, several researchers attempted to track, monitor, and predict students' performance to identify factors that could help undergraduate students or improve their performance [123]. In the same token, several researchers analyzed students' learning approach, navigational behavior, and the effect of learning design [159]. Evaluation of course work using innovative methods, tools, or assignment types may be the dominant theme in the assessment topic, in particular, using automated assessment tools, grading methods, and feedback [104, 116, 164]. The dangers of *plagiarism* and the concerns on *academic integrity* have been a concern in programming education for decades now and continues to loom over the profession with the evolution of AI [69, 190]. The research timeline is more of a continuous cat-and-mouse, elusive chase where researchers pursue methods that allow a fair assessment of students who put real effort in their assignments [163].

5.2.5 Advanced Topics. This category covers some of the programming paradigms beyond the basics of computing, as well as other more advanced topics.

OOP has been one of the most relevant paradigms in programming education for decades [173]. There is a longstanding debate about whether it should constitute the way novice students are introduced to programming—often through the Java programming language—or whether the focus should be first on the basic programming structures [8, 79, 106]. The research focus on OOP has traditionally revolved around students' understanding of the object-orientation concepts that differentiate this paradigm from others, how to present them in an easier way, and even in what order. Some examples of such concepts are objects and classes, inheritance, polymorphism, and design patterns [32, 72, 115, 175]. Recently, the focus has shifted to innovative pedagogy such as flipped classroom [22] and game-based learning [41], or tools like interactive tutorials [65].

Web Programming. With the rise of Internet access and advancements in browser technologies, what were previously static web sites became web applications. Therefore, learning HTML coding was no longer enough for CS majors. Although commonplace languages like Java were widely used for web development [122], other niche languages specifically tailored to the web began to appear in the CS classroom, such as PHP [98] or more recently, JavaScript and its server engine node.js [87], to cater to the job market needs. On a different vein, web-based learning environments for introductory programming also began to emerge as they removed the complexity of a real-life IDE and often incorporated automated assessment capabilities [68].

Software engineering fundamentals are crucial in computer science education as they help students situate their newly acquired programming skills within the broader scope of a complete software project. A large share of the existing research has focused on teaching software engineering methodologies, such as agile development [46, 67]. Other articles have focused on tools and pedagogical approaches for teaching specific aspects of software engineering, such as requirement elicitation [45], software modelling [26], and testing [73, 204]. Last, there is a growing interest in providing students with industry-like experience [29] by teaching code maintainability [160], "production" programming [7], or by contributing to open source software projects [150].

Mobile Programming. Using block-based environments such as App Inventor has been a common way to introduce children, novice programmers, and students with no technical background to application development [120, 201, 247]. Of course, the use of text-based programming languages has also been researched, mainly Android development [211, 212]. Last, the topic of mobile programming also encompasses low-level programming of wireless network functionalities [83].

Advanced Programming. More advanced topics in programming are, for instance, parallel and distributed computing, which constitute an important paradigm shift for students. Research has investigated how to make parallel and distributed computing understandable for students and how to integrate it into the curriculum [56, 60]. Among the topics covered are synchronization [47], multi-core architectures [112], and semaphores [37]. There has also been research on specific tools and programming environments for teaching parallel and distributed computing [62], as well as no-code environments [11]. More recently, a central focus has been on cloud computing and big data [129, 207].

Logic and formal presentation covers work which discusses formal logic (not logic programming) in the context of programming, for example, in program verification [97] or formal specification [39]. There is also much work discussing discrete mathematics and the role of mathematics in learning programming or computer science more broadly, e.g. [130, 153]. The category also includes much work related to functional programming, e.g. [49, 234]. A significant share of these works is carried out in introductory programming contexts. However, as the point of view differs radically from mainstream research in novice programming, we have included this topic among advanced topics. Notably, a major share of the papers in this topic are rather old, published in the 1970s–1990s.

AI, ML, and Modelling (Artificial Intelligence, Machine Learning, and Modelling). The last topic in this category encompasses a variety of research lines revolving around the notion of “model.” First and foremost, machine learning models have been used for predicting students’ success or learning struggles [169], which has been a common goal of learning analytics and educational data mining research. Second, an increasingly relevant topic is the use of large language models to assist students in coding [171, 243], which is a rapidly growing research topic in computing education. We expect this topic to continue growing in the coming years. Third, teaching and learning machine learning and AI have, of course, received attention in our dataset, especially within computational thinking studies that push for going beyond rule-based programming principles in computational thinking [216]. AI- and ML-related research heavily dominates this cluster, especially the last 5 years of research, while earlier research covers more varied topics addressing students’ understanding and mental models, especially on recursion and notional machines.

5.2.6 Digital and Physical Facilities. The topic of *Classroom and online activities* focuses mostly on blended learning contexts that involve learning both in the classroom and in digital environments. The flipped or inverted classroom methodology has received the most attention in the literature [9, 139] as an increasingly popular alternative to making the most out of class time. Another important focus has been the promotion of active learning among students with online environments that enable their active engagement [88]. Last, fully online environments have also received increasing attention, for example, programming MOOCs [161].

Learning systems and *Supporting tools* are two closely related and partially overlapping topics. They cover digital platforms that students use for learning, as well as a plethora of tools for learning programming that have been developed, covering diverse topics. Much attention has been paid to intelligent tutoring systems, especially for introductory programming, although some exist for teaching more advanced techniques [55, 238]. Another focus of research has been the integration of programming-specific tools into existing (generic) learning management systems [50, 85]. One important focus has been automated assessment tools [156, 164], which offer automated scoring and oftentimes automated feedback generation of programming assignments. Other tools exist for introducing basic programming concepts [182], program visualization [4], or algorithm visualization [188]. Tools for learning block-based programming have also received substantial attention [117].

Games and Gamification. Game development has been used as an underlying theme of CS courses in order to motivate students to gain interest in programming [124]. Moreover, games

have been used as tools for learning a wide range of topics [197], from the basics of computational thinking [102] to more advanced topics such as algorithms [249] or web programming [138]. Last, gamification has been used as a strategy to encourage students to participate more in the classroom, e.g., by using audience response systems like Kahoot [237], or to keep them engaged in remote learning settings, e.g., through the use of badges and rewards [44].

5.2.7 Investigation. Finally, we have two categories focusing on investigating students' behavior and success. *Psychological factors* encompass multiple psychological factors that are associated with learning programming on various course levels [142], such as students' attitude [240], self-efficacy [218], emotions [54], and perception of learning programming [165]. The latter one, which we labelled *Research on CS1*, covers quite different work related to students' progress and learning outcomes [149, 199], and here, papers heavily focus on introductory programming courses and novice programmers.

5.3 Topic Trends

Programming education, especially introductory programming education, has been a subject of research since the late 1960s. However, the volume of published works has exploded during the last 20 years. In our dataset, the annual number of papers published from the 1960s to the first years of the 2000s remains below 100. During the last 10 years, the number of papers has grown to several hundred, even roughly 800 in the top years 2019 and 2020. The largest topic clusters include *teaching*, *assessment*, *computational thinking*, and *STEM and curriculum*, each with well over 600 papers during the whole time span. The smallest clusters include *plagiarism and academic integrity*, *programming tasks*, and *supporting tools* with around 100 papers each. See Table 2.

Global publication rates have increased globally due to a multitude of factors: more publishers, emphasis on publishing metrics, and recent expansion of venues, as well as a spike during and after the COVID pandemic. These increases in the rate of productivity and rate of publishing cannot be decoupled from a genuine, meaningful rise in interest in the field. A comparison with other close fields shows an 11.4% rate of increase in CER as a whole, and 18% in computer-supported collaborative learning [111].

Figure 3 presents the growth of the number of papers in each of the themes (top line) and different topics under them. In all themes, the growth has been rapid in the last two decades. However, a closer look shows that papers focusing on *logic and formal presentation*, *programming languages*, *OO programming*, *advanced programming*, *novice coding process and errors*, *classroom and online activities*, *STEM and curriculum*, and *software engineering* have started gaining much attention already in the 1980s and 1990s. On the other hand, research on *plagiarism and academic integrity*, *teamwork*, *games and gamification*, *computational thinking*, *learning systems*, and *assessment*, as well as *AI*, *ML*, and *modelling* has been very little addressed before 2000.

When we normalize the numbers by dividing the number of publications in each category and theme by the total number of papers published in the same year, we get a quite different image of the popularity of the research topics (Figure 4). In most themes and categories, the activity seems to stay on quite the same level with small fluctuation only. However, some interesting trends can be observed. First, the share of advanced topics has steadily decreased after 1990s. There also seems to be a small decrease in the theme CS1 and CS2, which seems contradictory. However, as noted above, many other themes cover also introductory programming research from different perspectives.

When looking at the individual categories, we see that there was a boom of research activity concerning *OO programming* from 1990s till around 2010. This obviously reflects the emergence of C++ and Java and OO programming in CS1/CS2 and likely also the sometime heated debates on objects-first and objects-last approaches. Second, there is clear decline of interest in *Logic and*

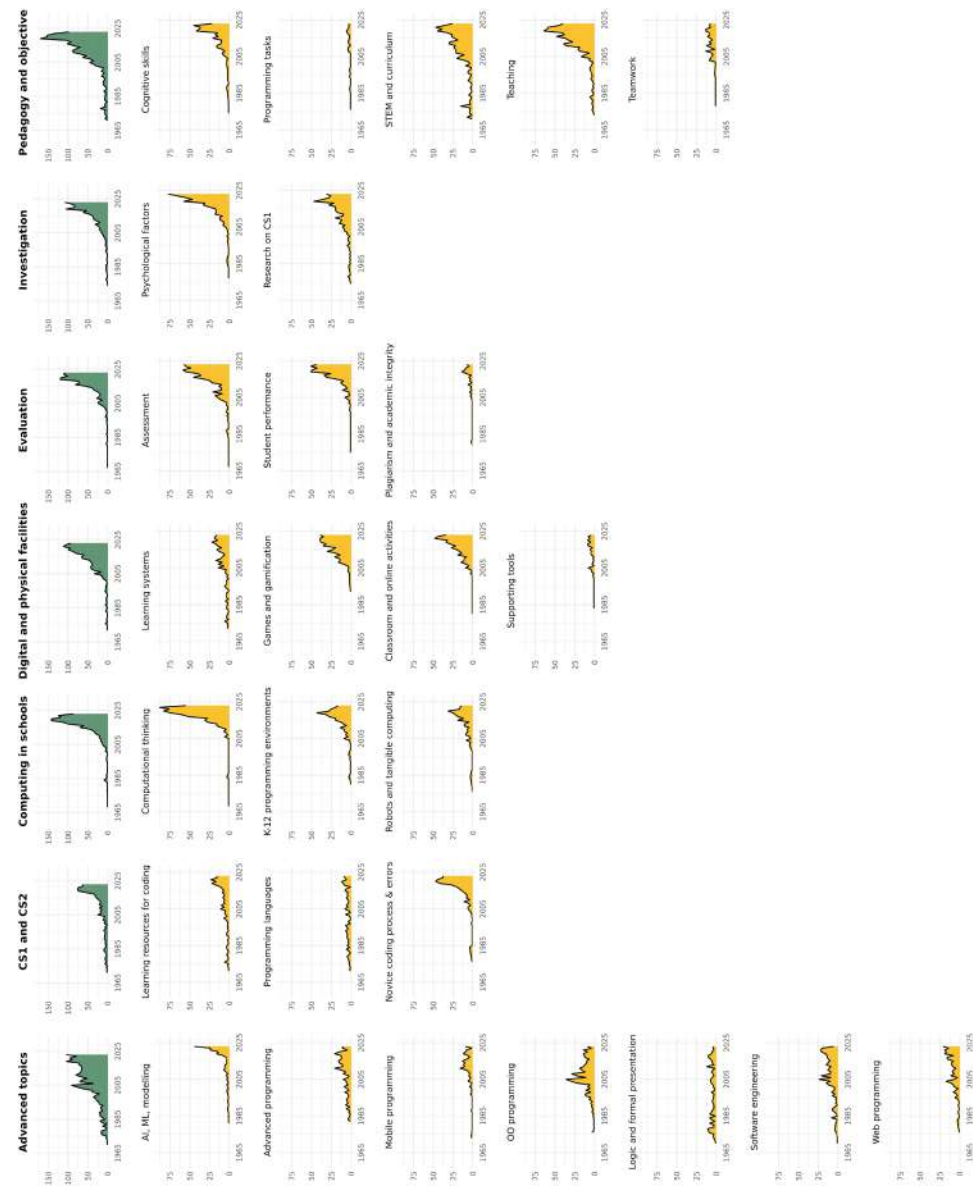


Fig. 3. Frequency of articles that have each topic as their top (highest gamma).



Fig. 4. Percentage of articles from total published papers that have each topic as their top (highest gamma). Note that the total number of publications in the early years was small, and therefore, the fluctuation and single peaks can be related to a very small number of papers in the categories.

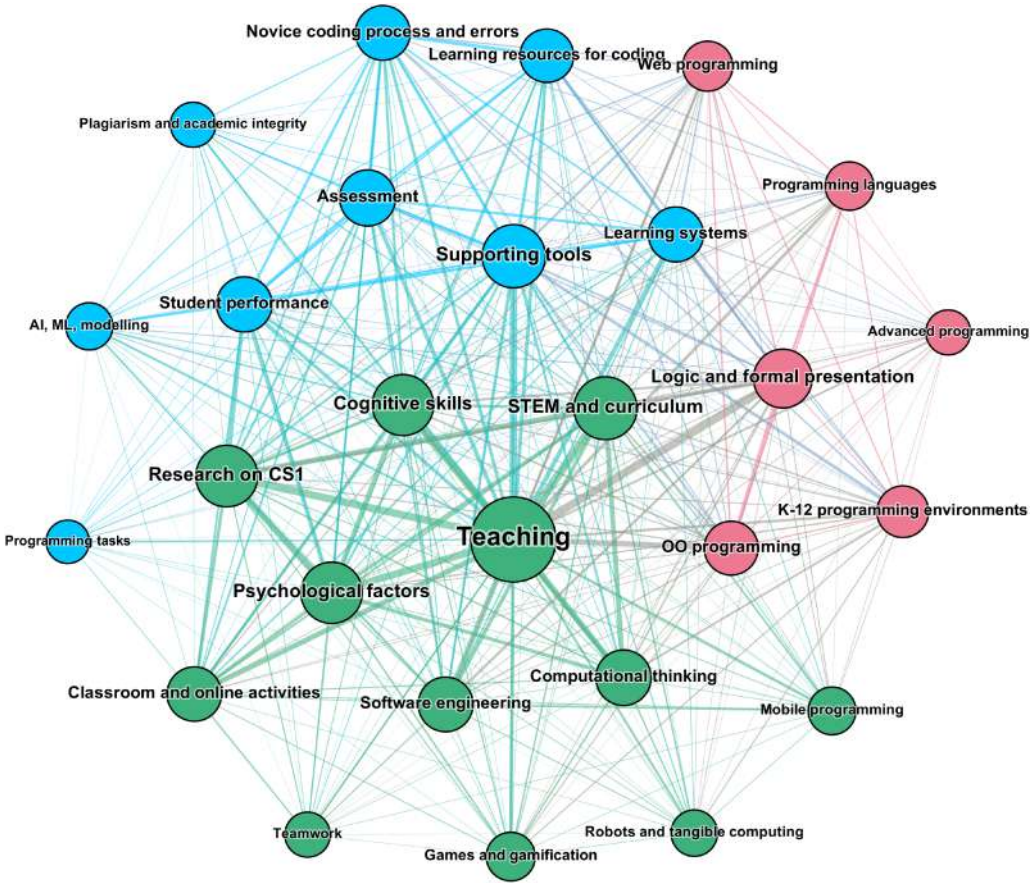


Fig. 5. Topic co-occurrence.

formal presentation. The research in this area was very active in the early years, but its share in the data pool has steadily decreased. A similar trend is seen with *Programming languages*. We do not see a strong growth in any areas. However, a closer look reveals clear growth of activity in *AI, ML, and modelling* in the last years, as can be expected. There is growth also in *Computational thinking*, *Student performance*, and *Psychological factors*.

5.3.1 Topic Co-Occurrence. The network of co-occurrence of topics (Figure 5) is rather heterogeneous and is characterized by dense interconnections between most topics, pointing to the diversity and interdisciplinarity thereof. In topic modelling, each paper could be associated with several topics, and the topmost topic based on gamma was selected to denote the paper category, as presented above. However, in this visualization, the five topmost topics associated with a paper are used to denote the connections between different topics.

Three clusters can be identified: a blue cluster with topics that are dominated by topics about organization of programming education, e.g., *learning systems*, *supporting tools*, *students' performance*, *assessment*, *plagiarism*, and *academic integrity*. The largest cluster in terms of the prevalence of topics and strong connections is the green cluster. In the heart of the green cluster lies the *teaching* topic, connected strongly with *STEM and curriculum*, *classroom and online activities*, and *cognitive skills*. Other topics within this cluster include *psychological factors* and *research on CS1* covering

Table 3. Top 25 Publication Venues with at least 50 Programming Education Papers

Conference/Journal	Count	Percentage
ACM TECHNICAL SYMPOSIUM ON COMPUTER SCIENCE EDUCATION, SIGCSE	1,259	12.8%
INNOVATION AND TECHNOLOGY IN COMPUTER SCIENCE, ITICSE	751	7.6%
IEEE FRONTIERS IN EDUCATION CONFERENCE, FIE	465	4.7%
ACM SIGCSE BULLETIN	431	4.4%
ASEE ANNUAL CONFERENCE AND EXPOSITION	248	2.5%
INTERNATIONAL CONFERENCE ON EDUCATIONAL RESEARCH, ICER	222	2.3%
KOLI CALLING INTERNATIONAL CONFERENCE ON COMPUTING EDUCATION RESEARCH	192	1.9%
AUSTRALASIAN COMPUTING EDUCATION CONFERENCE, ACE	159	1.6%
COMPUTER SCIENCE EDUCATION	142	1.4%
ACM TRANSACTIONS ON COMPUTING EDUCATION	129	1.3%
IEEE GLOBAL ENGINEERING EDUCATION CONFERENCE, EDUCON	105	1.1%
WORKSHOP IN PRIMARY AND SECONDARY COMPUTING EDUCATION, WIPSC	104	1.1%
INTERNATIONAL CONFERENCE ON INFORMATICS IN SCHOOLS, ISSEP	91	0.9%
INTERNATIONAL CONFERENCE ON COMPUTERS IN EDUCATION, ICCE	79	0.8%
IEEE TRANSACTIONS ON EDUCATION	72	0.7%
ANNUAL CONFERENCE ON INFORMATION TECHNOLOGY EDUCATION, SIGITE	71	0.7%
EDUCATION AND INFORMATION TECHNOLOGIES	69	0.7%
INTERNATIONAL CONFERENCE ON COMPUTER SUPPORTED EDUCATION, CSEDU	69	0.7%
IEEE INTERNATIONAL CONFERENCE ON ADVANCED LEARNING TECHNOLOGIES, ICALT	66	0.7%
IEEE SYMPOSIUM ON VISUAL LANGUAGES AND HUMAN-CENTRIC COMPUTING, VL/HCC	66	0.7%
INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING, ICSE	65	0.7%
ACM INROADS	61	0.6%
COMPUTER APPLICATIONS IN ENGINEERING EDUCATION	57	0.6%
COMPUTERS AND EDUCATION	57	0.6%
INTERNATIONAL CONFERENCE ON COMPUTER SCIENCE AND EDUCATION, ICCSE	52	0.5%

The percentage is the share of papers in the total paper pool.

commonly studied teaching-related topics. Moreover, notably, there are relatively new topics such as *robotics* and *mobile programming*. The last cluster, the smallest in number of topics, is the red cluster, which is dominated by programming curricular strands and traditions, e.g., *OO programming*, *web programming*, and *advanced programming*. To summarize, the field of programming research can be roughly conceptualized as three interconnected major themes that are teaching, organization, and implementation of teaching, and curricular strands with significant overlap.

5.3.2 Topic Activity in Journals and Conferences. Programming education-related papers are published in a vast number of venues. Our dataset, including 9,865 papers, has been published in 1,959 different venues.⁴ Almost all of these are journals and conference series, and only a very small number of papers have been published in book chapters or complete books (see also Table 1). The most important venues are SIGCSE-related conferences (SIGCSE Technical Symposium, ITiCSE, ICER, and CompEd), covering 22.9% of papers. Major engineering education conferences, IEEE Frontiers in Education, ASEE Annual conference, and EDUCON cover 8.3% of papers. A list of venues with more than 50 papers in our dataset is given in Table 3.

Notably, the dataset includes 1,254 venues with only one paper about programming education. Almost one-third of the papers (3,225) have been published in venues where the total number of programming education papers is less than 10 papers.

⁴For clarity: Proceedings of the same conference in different years are here considered as a single venue, e.g. SIGCSE Technical Symposium is only one venue of the almost 2,000 venues.

Table 4. Publications in SIGCSE-Associated Venues and Other Publication Venues over the Years

Years	SIGCSE		Other		Total Count
	Count	%	Count	%	
1967–1979	150	76.9%	45	23.1%	195
1980–1989	245	68.6%	112	31.4%	357
1990–1999	302	59.0%	210	41.0%	512
2000–2009	651	39.1%	1,013	60.9%	1,664
2010–2019	992	23.2%	3,277	76.8%	4,269
2020–2023	532	18.6%	2,336	81.4%	2,868
Total	2,872	29.1%	6,993	70.9%	9,865

In Table 4, we compare the number of publications in SIGCSE-associated conferences (SIGCSE Technical Symposium, ITiCSE, ICER, CompEd) as well as closely related journals/magazines SIGCSE Bulletin, ACM Inroads, and ACM TOCE, and all other publication venues over the period in our data, counting the share of publications of total published papers in each decade. SIGCSE and related venues dominated the field till the millennium, but thereafter the publication volume in other venues has grown much higher, thus demonstrating the growing breadth of research. A detailed analysis of the almost 2,000 publication venues, however, is out of the scope of this article. A great share of these are various technical computing venues, but programming papers are published in many other fields, as well, especially in engineering education and venues focusing on using computers in education.

Figures 6 and 7 present an overview of how various topical areas are present in several key conferences and journals that focus on only computing education, with a few engineering education venues, where programming education is heavily present. The darker color indicates a stronger presence of the topic among the data pool for each specific publication venue. As we can see, *computational thinking* is very heavily present in two conferences, ISSEP and WIPCSE, both of which focus on K-12 computing education. Many conferences have certain topics which are well present there: ASEE (*STEM and curriculum, research in CS1, software engineering*), FIE (*research in CS1, teaching and STEM and curriculum*), ICER (*Research in CS1, psychological factors, student performance, novice coding process and errors, cognitive skills*), ICSE-SEET (*assessment, novice coding process and errors, learning resources for coding*), SIGITE (*research in CS1, teaching and STEM and curriculum*). On the other hand, major computing education conferences ITiCSE and SIGCSE technical symposium cover widely many topical areas, and they also have stronger coverage of *advanced programming* topics, which are rarely present in ICER, ICSE-SEET, and Koli.

From another perspective, some topics are quite evenly covered in all conferences: *supporting tools, classroom and online activities, cognitive skills, psychological factors*, and *OO programming*. In general, rarely addressed topics include *robots and tangible computing, AI, ML and modelling, advanced programming, programming languages, teamwork, mobile programming*, and *programming tasks*.

Considering journals and magazines, *teaching and computational thinking* are highly present in Informatics in Education, *logic and formal presentation* in SIGCSE Bulletin, *psychological factors* and *cognitive skills* in Computer Science Education, and *STEM and curriculum* in ACM Inroads. Widely addressed topics in all venues include *STEM and curriculum, teaching and research on CS1*. On the other hand, rarely addressed topics are *robots and tangible computing, web programming, AI*,

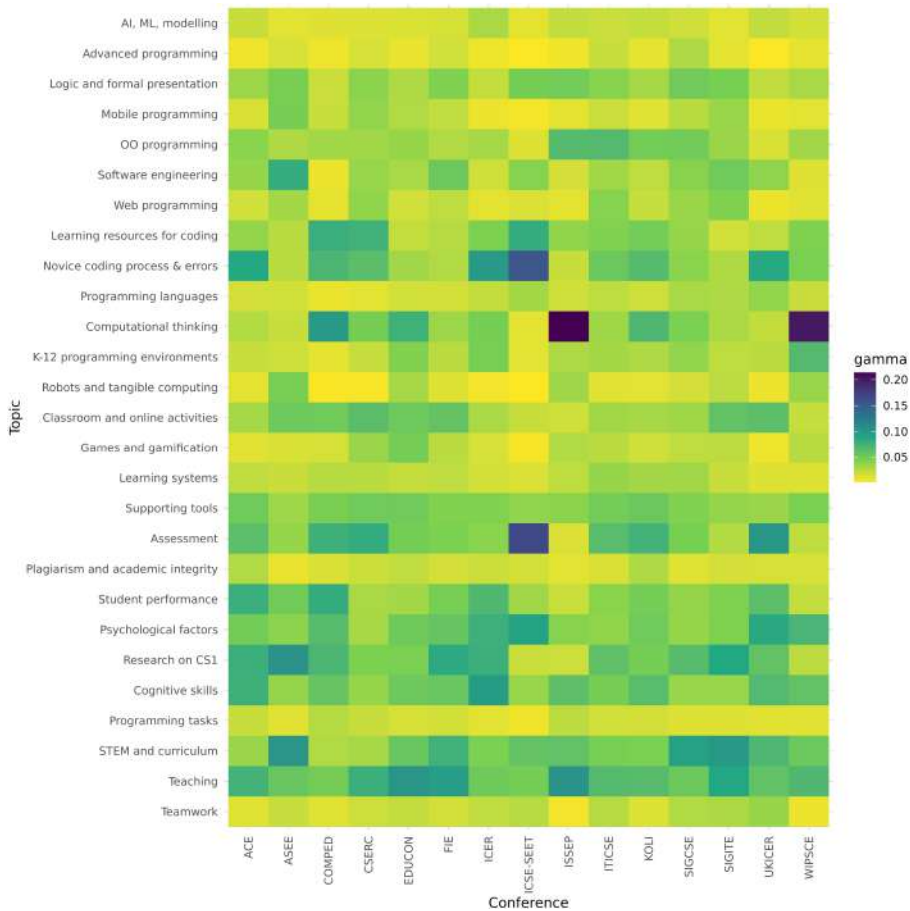


Fig. 6. Heatmap of topic occurrence in each of the main computer science education conferences.

ML and modelling, games and gamification (except in Informatics in Education), team work, mobile programming, and programming tasks.

Both ACM Inroads and SIGCSE Bulletin have quite similar profiles, which is not a surprise, as they denote the same venue with a name change (and profile update).⁵ When comparing ACM TOCE and Computer Science Education, the first one has a somewhat more balanced profile over all themes, not too much different from IEEE Transactions on Education. Informatics in Education has a somewhat different profile.

6 Discussion

6.1 Growth of the Field

While research in programming education has a very long tradition, starting from the late 1960s, the volume of the work in terms of published papers started a rapid growth only some 20 years ago, resulting in almost 10-fold of papers published in the late 2010s. Interestingly, the main reason for this growth has not been the launch of CER-focused conferences, such as the International

⁵After ACM Inroads was launched in 2010, SIGCSE Bulletin is still published as an electronic newsletter. In our dataset, SIGCSE Bulletin denotes the venue before the transition.

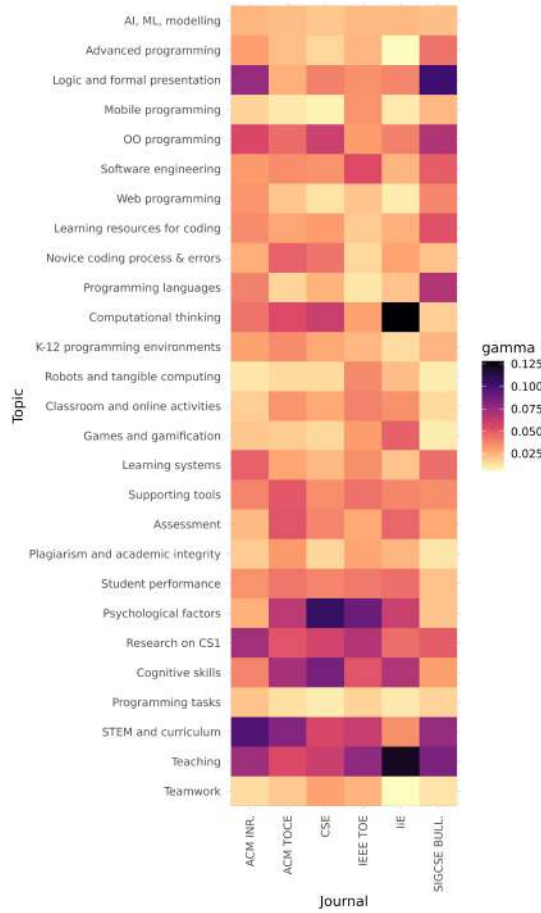


Fig. 7. Heatmap of topic occurrence in each of the main computer science education journals.

Computing Education Research conference (ICER; 2005 onward) and Koli Calling (since 2001) [16]. As shown in Table 4, SIGCSE-related publications venues have grown significantly, and the total publication count has more than tripled since the 1990s. However, at the same time, the number of papers published in other venues has grown well more than 10-fold.

The massive growth can be attributed to several factors. The ICT industry and applications of computing in all fields in science and society have grown rapidly, building the need to extend the volume of computing education in universities globally. Another reason for rapid growth is research on computational thinking, which has been boosted by the decisions of many countries to include computing, programming, or CT in their school curricula, as well as the renaissance of the concept after the famous position paper by Wing [245]. Finally, there is an expansion of publishing as an industry with several major publishers in the last two decades.

6.2 Topical Trends

The variation of topics provides a rich picture of the field and demonstrates the growth of the field, not only in the number of papers, but also in the breadth of work in terms of curricular content and points of view. While work in introductory programming is the most active research area from

a curriculum point of view, our analysis also identified several other curricular topics, covering mobile programming, web programming, OO programming, advanced programming (especially parallel, concurrent, and distributed programming, as well as software engineering and AI/ML). On the other hand, several curricular topics were not visible in the topic modelling, despite that programming is related to them, such as data structures, databases, operating systems, embedded systems, compilers, and security. However, when scanning the titles and abstracts, these terms were visible, but their numbers were counted in a few dozen at most. Most papers addressing data structures were within *learning systems* topic, which matches with algorithm visualization research. Program visualization papers were also mainly in the same topic (only a small minority of these papers were in *supporting tools*, thus indicating a difference in these clusters). Moreover, operating systems papers were mostly in learning systems, as well. Security-related papers were heavily focused on *software engineering* topics. All these demonstrate that there is work in these areas, but its volume is small.

As noted, introductory programming is the largest addressed topical area. However, there is a great variation of points of view present in the data. There is work covering logic and formal presentation, learning resources, programming process and students' errors, choice of programming languages, K-12 programming, robots and tangible programming, classroom and online activities, games and gamification, learning systems and supporting tools, assessment, plagiarism, student performance, psychological factors, and other research on various factors influencing novice students' working, cognitive skills, programming tasks, teaching, and team working. In all clusters, there are naturally papers addressing also other courses than CS1/CS2. As a whole, this reveals a very rich image of the research.

When we look at the growth rates in different themes and clusters in Figure 3, we can recognize several areas that have become highly popular in the last 20 years, starting to grow around 2005. First, research on *Computational thinking* has become one of the most popular subareas, obviously reflecting the growth of computing in schools. Second, research on *psychological factors* has been booming. This is likely connected to the development of the CER field as a whole in terms of building a more rigorous and theory-based understanding of factors that influence student success and behaviors. Strong growth can also be seen in *students' performance* and *assessment*, which fit together with it. Third, work in *novice coding process and errors* has also expanded rapidly, which reflects the growing interest in learning analytics research. Fourth, there is also strong growth in areas related to pedagogy and supporting facilities, which can be seen in areas *teaching*, *cognitive skills*, *games and gamification*, and *classroom and online activities*. Fifth, just in recent few years, research in the area *AI, ML, and modelling* has skyrocketed, especially reflecting the work on novel AI tools based on large language models. Finally, there is moderate growth in advanced topics. For example, *advanced programming* cluster includes a large number of papers addressing parallel, concurrent, and distributed programming. In addition, *software engineering* and *web programming* have shown steady growth.

While these areas have been addressed heavily, there are several areas where we see very little growth or even decreasing publication activity. For example, work related to the choice of *programming languages* does not show any significant growth. This is no surprise, as changes in programming languages used in curricula are slow. *Logic and formal presentation* was an active area (compared to total research volume) in the 1970s and 1980s, but it has not grown at all, indicating that these methods are nowadays rarely used in programming education, which has grown very much in the following decades. The same cluster also covered many papers on functional programming and discrete mathematics connected to programming courses. Correspondingly, their presence in the field is low. Finally, there is only little growth in *learning systems* and *supporting tools* despite that tools research has always been a relevant part of CER. Research on *OO programming* was

most active around 2005, but thereafter it has received much less attention. This is interesting and probably reflects that major issues have been resolved. For example, 20 years ago, there were heated debates on objects-first and objects-late approaches [128], but the choice is not a big issue anymore.

The relative trends of topical areas, measured as the share of papers in a particular area divided by the total publication volume at the same time, as depicted in Figure 4, reveal some significant observations. The share of research in *Advanced topics* has consistently declined for a long time. This results naturally due to the rapid growth of research in all other themes, especially *Computing in schools*. On the other hand, when examining the relative trends as a whole, the development is quite balanced. There are very few significant changes after the year 2000. In the early years before that, there was more fluctuation, but then the total number of published papers was much smaller, and therefore, a small number of papers in some area could change the big picture of the field.

6.3 Other Observations

One of the notable observations is that the term “theory” is present only once among the term lists of the clusters. This suggests that theory is not among the dominating features of programming education research papers. On the other hand, theories have only quite recently received growing interest in CER [141, 143, 213, 214], and thus, they may not be visible enough in our dataset. Another reason is likely that terms such as “theoretical framework” or a name of some theory, e.g., constructionism or cognitive load theory, may have been used instead, and these are not visible in our results. Interestingly, the term “model” appears only in the term list of *AI and ML* cluster, even though modelling is one of the key goals of computing methods. On the other hand, “design” appears in multiple term lists, indicating its strong presence in the dataset.

When we consider the relation of topics in Figure 5, one immediate observation is that the network is very tightly connected. This is not a surprise, as topic modelling allows identifying several topics within a single topic. We split the papers into separate clusters based on the highest gamma value for the paper. In the figure, however, each paper was associated with five topmost topics, and the heavy overlap is clearly visible with thick edges between many topics. On the other hand, a closer look reveals many thin lines, e.g., between *teamwork* and *games and gamification*, *AI, ML, and modelling*, and *programming tasks* or *teamwork* and *mobile programming*, to give just a few examples. We also observe that the advanced topics, *advanced programming*, *AI, ML, and modelling*, *mobile programming*, *software engineering*, and *web programming* have links with most other topics, clearly denoting that they are not isolated areas of research.

Some of the salient areas in CER broadly were not as apparent in our analysis of programming education research [19]. One of such topics was that of gender and diversity. It appears that research around that topic revolves around choosing computer science as a major and increasing participation in computer science and STEM in general, rather than about programming specifically, and therefore, it was not evident in our findings.

The results concerning where different types of work are published (Figures 6 and 7) reveal clear differences between venues. It is very obvious that WIPCSE and ISSEP heavily emphasize computational thinking, as their initial goal is to solicit research in secondary education. On the other hand, the heatmaps reveal that papers on many other topics are published there, as well. However, a general level view seems to be that the other conferences have fairly similar profiles, except that ICSE SEET has a strong presence of topics *novice coding process and errors* and *assessment* compared to others.

Concerning journals and magazines, ACM Inroads and SIGCSE Bulletin have similar profiles, while the journals each have somewhat different emphasis, with Computer Science Education strong in *cognitive skills* and *psychological factors* and Informatics in Education in *computational*

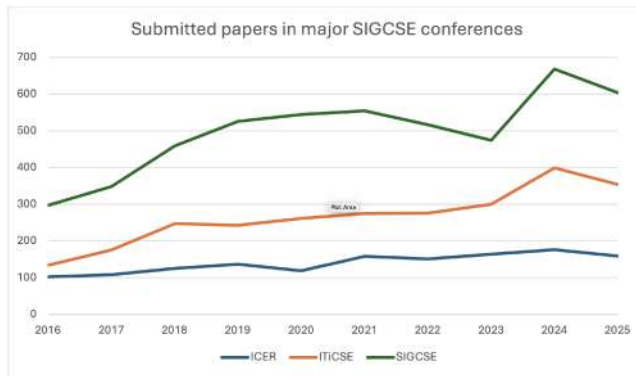


Fig. 8. Submitted full papers in major SIGCSE conferences.

thinking and *teaching*. IEEE Transactions on Education and ACM Transactions on Computing Education have somewhat balanced coverage of topics, although not equally.

6.4 Implications

Our results paint a broad image of the vast amount of work that has been published about programming education. The CER community has a strong concentration in the ACM SIGCSE community with associated venues. This is understandable as the conferences under this umbrella focus solely on computing education.⁶ On the other hand, the number of submissions in these conferences has significantly increased during the last 10 years, as shown in Figure 8. Despite some fluctuation, the number of submissions to the SIGCSE Technical Symposium has doubled, to ITiCSE almost tripled, and to ICER increased well over 50%. Correspondingly, the acceptance rates have been going down, to less than 20% in ICER, less than 30% in ITiCSE, and around 30% in SIGCSE.⁷

The increasing competition in getting papers accepted is likely to cause problems, especially for newcomers in the field, such as PhD students and CS teachers who wish to start writing papers about computing education. Therefore, building awareness of the breadth of publication venues is valuable. We hope that the lists of conferences and journals (Table 3 and Appendix A), which actively publish relevant work, some of them among other topics, are useful for readers when considering possible publication venues. It is more difficult to judge what is the value (for researchers in CER) of those hundreds of venues that publish relevant papers very infrequently. Skimming through titles in venues that only sporadically publish relevant papers may not seem reasonable. Search engines, of course, can find most of such papers. However, it may well be that the results are shown far from the top papers found, and thus may stay unnoticed. Such papers are naturally rarely cited.

Another perspective of the results is the relative share and growth of publication volume in different topical areas. We identified several hot topical areas where the publication volume has grown rapidly. From the researcher's perspective, working in a hot area is tempting as there is general interest in the topic, and as a bonus, the possibility of getting many citations for one's papers. A good example is the research on large language models in programming education, where the number of published papers and citation counts, especially on early frontline papers, has skyrocketed.

⁶In Appendix A, we list many other CER-focused venues, where programming education papers have been published but which did not make it to the top 25 list.

⁷Source: Frontmatter information in conference proceedings in ACM Digital Library.

Naturally, a limiting factor is often the availability of funding, and this is not necessarily correlated well with the publication counts. For example, while it is plausible that the general interest in incorporating and developing computing education in schools has brought more funding available to research in computational thinking in many countries,⁸ it is less clear whether the strong interest in psychological factors is based on more funding in this area. Historically, the application of educational sciences and psychology in CER has been limited in both quality and quantity [144]. More recently, this has been changing due to the growing interest of applying theoretical frameworks from social sciences in CER, e.g., [125, 143, 148, 220]. It is likely that more funding on the K-12 level computing education falls in this area, too.

On the other hand, it is not generally a good thing that some narrow research areas dominate the field. There are plenty of valuable research questions in all subfields waiting for someone to explore them. While there is little growth visible in many areas, such as *Mobile programming*, *Logic and formal presentation*, *Programming languages*, *Plagiarism and academic integrity*, or *Teamwork*, it would be unacceptable to claim that these areas in computing education should be ignored as research topics. On the contrary, for example, AI tools have created novel major challenges for academic integrity, which must be addressed. From the history of computing, we know that new programming languages will emerge and replace current languages in introductory programming. Logic, formal presentations, and theoretical computer science have always been among the core CS topics, while many students struggle in learning them [80].

When looking at the trends ourselves, we can propose some other recommendations for future research.

- There is ample and versatile work carried out in school-level programming, computational thinking, and first-year university courses, but much less focusing on the follow-up courses. However, multiple perspectives, such as psychological factors, cognitive skills, teaching, teamworking, and assessment, provide ample opportunities for new research in these contexts.
- Much programming education for adults is carried out outside formal educational settings in continuing or complementary education or informal education. For example, MOOCs provide ample opportunities for learning programming. However, in our data, there seems to be little research on them. We found only a few dozen papers mentioning MOOCs in titles or abstracts, mainly in the areas *assessment* and *classroom and online learning*.⁹

Finally, journal editors and conference chairs may consider our results when deciding on topics mentioned in calls for papers or updates in journal profiles. Are there some topical areas that should gain more visibility? Is the target profile of the venue well balanced? Should special issues in journals be targeted to some topical areas to encourage more work therein?

6.5 Limitations

Our decision to use the Scopus database for this research was motivated by the rich metadata that Scopus provides and its wide coverage of conferences and journals. Google Scholar would certainly have a broader coverage, but its metadata is thin and inconsistent compared to Scopus. With this decision, we also had to accept the limitations that Scopus has. For instance, Scopus does not include papers from all years of relevant conferences, e.g., Koli Calling. Moreover, as it is frequently updated with new information, it lags somewhat with recently published work, especially for conferences. Thus, when our search was carried out in spring 2024, all the data from

⁸For example, President Obama's *Computer Science For All* initiative in the USA. <https://obamawhitehouse.archives.gov/blog/2016/01/30/computer-science-all>.

⁹MOOC data have been heavily utilized in learning analytics and educational data mining research, but the research goals are typically more generic than improving computing education specifically.

2023 were not yet there. However, these shortcomings are not likely to have a great effect on building the big picture of the field, which is our goal.

We considered augmenting the existing Scopus data with two additional sources, ACM DL and IEEE Xplore. However, as the metadata from these databases were highly different from that of Scopus, it would be challenging to perform in-depth analyses. The lack of many metadata elements in IEEE Xplore and ACM Digital Library, which were available in Scopus, made it difficult to compile and analyze the data as a whole. In addition, there was a large paper overlap between Scopus and the other two databases, as Scopus also acquired many publications from these sources. Therefore, it was decided that Scopus was our only data source.

The part of the search process, which was based on keyword search from titles, abstracts, and author keywords, likely failed to identify all papers that would have been relevant for this analysis. In addition, programming education papers are also published in languages other than English, which we were not looking at specifically. However, the total data pool of close to 10,000 papers is very large, and we do not assume that omitted papers would change the big picture much.

The data cleaning process was a major effort. The decision to include all papers from computing education-focused conferences may have included in the dataset papers that are not actually addressing computing education, thus resulting in false positives. Some of these conferences may have accepted a broader selection of topics than we assumed. However, when we filtered the dataset to identify programming education papers only, we likely filtered most of such false positives out. During the analysis phase, when we skimmed through paper titles within the clusters, we identified very few possible false positives, concluding that more comprehensive manual checking of all data was not needed.

When cleaning the author data, we used Scopus author IDs as the keys. We acknowledge that Scopus provided dual IDs for some researchers, but these cases have little overall effect on the results of this article, and we made an effort to disambiguate these cases.

The manual part of the data cleaning process based on titles and abstracts was mostly very clear in terms of decisions to include or exclude papers. The less obvious cases were discussed by two researchers together until a consensus was reached. However, there were many papers where a more detailed policy was needed, as written above in Section 4. Decisions on several papers were not clear-cut, but their share in the total data pool is insignificant.

The final dataset with almost 10,000 papers was considered far too large for any systematic manual categorization. We therefore decided to use scientometric methods, especially topic modelling, for building categories. Structured topic modelling is a machine learning method that produces a number of clusters. When looking at the cluster profiles with different total numbers of clusters, it became very obvious that many clusters, e.g., *software engineering* or *plagiarism and academic integrity* were clearly present regardless of whether the total count of topics was in the range 20–30. On the other hand, some topics were not initially very clear, especially as papers can address introductory programming from different points of view. The decision to accept 27 topics was made with the consensus of three authors. A follow-up process of skimming through a large share of titles in each cluster and checking the presence of relevant keywords revealed clear differences in the cluster characteristics, and we revised the initial labels to better reflect these aspects. However, we acknowledge that there is some inherent overlap of the categories, which is not possible to remove due to the nature of the data, human-authored papers. The topic names and the wider theme names reflect our best understanding of the richness of topic contents and perspectives.

7 Conclusion

This research has provided a comprehensive thematic review of the programming education research field spanning almost 60 years and taking into account roughly 10,000 dedicated papers.

We detailed a data retrieval framework that allowed us to distill a quality set of papers from a vast and heterogeneous database, Scopus. From this dataset, we further utilized keyword analysis and topic modelling to identify over 27 research topics, which were grouped into 7 core themes representing the main directions in programming education research.

With these data, we gave an overall description of work in each topical area, also giving a number of sample papers in each area. In addition, we explored how research activity in the topical areas has evolved and what kind of connections there are between the areas. Our findings show that during the first decades till the millennium, programming education research gained steady but quite slow growth. However, during the last 20 years, publication activity in the field has rapidly grown. Most of this growth results from papers that are published elsewhere than in the ACM SIGCSE-associated venues, which are solely targeted to CER. This apparently reflects the wide interest in programming education and computational thinking in other fields, such as technical computer science venues, engineering education, science education, and general education. The largest theme in the data is Advanced topics, followed by Pedagogy and objectives, and Digital and physical facilities. However, a large number of papers in most themes also discuss introductory programming, but from different points of view. We conclude that research on first-year programming is the most common topic, which is investigated from many different perspectives that cover many different topical areas. We could have selected another, smaller number of categories with less overlap. However, the current version better describes the richness of the field.

The Advanced topics theme is large, and most of these papers indeed focus on more advanced topics than first-year programming, an observation which is often shadowed by the strong presence of papers focusing only on basic things. We are delighted by this observation and encourage further work and more visibility in this advanced area.

We analyzed what kind of profiles key publication venues, including conferences and journals, have in terms of their coverage and focus on different topical areas, identifying significant differences in the profiles. We hope that such information will be useful for newcomers to the field, who can better identify the most relevant venues for submitting their papers. On the other hand, journal editors and conference program chairs, and steering groups may use this information to tune and possibly redirect and revise the lists of topics that the venue hopes to solicit.

References

- [1] [n.d.]. Curricula Recommendations. Retrieved June 11, 2024 from <https://www.acm.org/education/curricula-recommendations>
- [2] Alfred V. Aho. 2011. Ubiquity symposium: Computation and computational thinking. *Ubiquity* 2011, Article 1 (January 2011), 1–8. DOI: <https://doi.org/10.1145/1922681.1922682>
- [3] Alfred V. Aho. 2012. Computation and computational thinking. *The Computer Journal* 55, 7 (July 2012), 832–835. DOI: <https://doi.org/10.1093/comjnl/bxs074>
- [4] Abdullah Al-Sakkaf, Mazni Omar, and Mazida Ahmad. 2019. A systematic literature review of student engagement in software visualization: A theoretical perspective. *Computer Science Education* 29, 2–3 (2019), 283–309. DOI: <https://doi.org/10.1080/08993408.2018.1564611>
- [5] Ali Alammery. 2019. Blended learning models for introductory programming courses: A systematic review. *PloS One* 14, 9 (2019), e0221765.
- [6] Ibrahim Albluwi. 2019. Plagiarism in programming assessments: A systematic review. *ACM Transactions on Computing Education* 20, 1 (2019), 1–28.
- [7] Eric Allen, Robert Cartwright, and Charles Reis. 2003. Production programming in the classroom. *ACM SIGCSE Bulletin* 35, 1 (2003), 89–93. DOI: <https://doi.org/10.1145/792548.611940>
- [8] Nabeel Alzahrani, Frank Vahid, Alex Edgcomb, Kevin Nguyen, and Roman Lysecky. 2018. Python versus C++: An analysis of student struggle on small coding exercises in introductory programming courses. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education (SIGCSE '18)*. ACM, New York, NY, 86–91. DOI: <https://doi.org/10.1145/3159450.3160586>

- [9] Ashish Amresh, Adam R. Carberry, and John Femiani. 2013. Evaluating the effectiveness of flipped classrooms for teaching CS1. In *2013 IEEE Frontiers in Education Conference (FIE)*. IEEE, 733–735. DOI : <https://doi.org/10.1109/FIE.2013.6684923>
- [10] Boyd Anderson, Martin Henz, Kok-Lim Low, and Daryl Tan. 2021. Shrinking JavaScript for CS1. In *Proceedings of the 2021 ACM SIGPLAN International Symposium on SPLASH-E (SPLASH-E '21)*. ACM, New York, NY, 87–96. DOI : <https://doi.org/10.1145/3484272.3484970>
- [11] Barbara M. Anthony, D. Cenk Erdil, Olga Glebova, and Robert Montante. 2022. Unplugged parallelism for first-year CS majors. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education (SIGCSE '22)*. ACM, New York, NY, 1079. DOI : <https://doi.org/10.1145/3478432.3499051>
- [12] Mikko Apiola, Matti Lattu, and Tomi A. Pasanen. 2010. Creativity and intrinsic motivation in computer science education: Experimenting with robots. In *Proceedings of the 15th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '10)*. ACM, New York, NY, 199–203.
- [13] Mikko Apiola, Matti Lattu, and Tomi A. Pasanen. 2012. Creativity-supporting learning environment—CSLE. *ACM Transactions on Computing Education* 12, 3 (July 2012), Article 11, 1–25. DOI : <https://doi.org/10.1145/2275597.2275600>
- [14] Mikko Apiola, Sonsoles López-Pernas, and Mohammed Saqr. 2023. *Past, Present and Future of Computing Education Research: A Global Perspective*. Springer Nature.
- [15] Mikko Apiola, Sonsoles López-Pernas, and Mohammed Saqr (Eds.). 2023. The venues that shaped computing education research: Dissemination under the lens. In *Past, Present and Future of Computing Education Research: A Global Perspective*. Springer, 121–150.
- [16] Mikko Apiola, Sonsoles López-Pernas, Mohammed Saqr, Arnold Pears, Mats Daniels, Lauri Malmi, and Matti Tedre. 2022. From a national meeting to an international conference: A scientometric case study of a Finnish computing education conference. *IEEE Access* 10 (2022), 66576–66588.
- [17] Mikko Apiola, Mohammed Saqr, and Sonsoles López-Pernas (Eds.). 2023. The evolving themes of computing education research: Trends, topic models, and emerging research. In *Past, Present and Future of Computing Education Research: A Global Perspective*. Springer, 151–169.
- [18] Mikko Apiola, Mohammed Saqr, and Sonsoles López-Pernas (Eds.). 2023. The hands that made computing education research: Top authors, networks, collaboration and newcomers. In *Past, Present and Future of Computing Education Research: A Global Perspective*. Springer, 101–119.
- [19] Mikko Apiola, Mohammed Saqr, Sonsoles López-Pernas, and Matti Tedre. 2022. Computing education research compiled: Keyword trends, building blocks, creators, and dissemination. *IEEE Access* 10 (2022), 27041–27068. DOI : <https://doi.org/10.1109/ACCESS.2022.3157609>
- [20] Mikko Apiola and Erkki Sutinen. 2021. Design science research for learning software engineering and computational thinking: Four cases. *Computer Applications in Engineering Education* 29, 1 (2021), 83–101. DOI : <https://doi.org/10.1002/cae.22291>
- [21] Golnaz Arastoopour Irgens, Sugat Dabholkar, Connor Bain, Philip Woods, Kevin Hall, Hillary Swanson, Michael Horn, and Uri Wilensky. 2020. Modeling and measuring high school students' computational thinking practices in science. *Journal of Science Education and Technology* 29, 1 (2020), 137–161. DOI : <https://doi.org/10.1007/s10956-020-09811-1>
- [22] Pasquale Ardimento and Michele Scalera. 2021. Experimentation of flipped learning in a university course on object-oriented programming paradigm. In *Communications in Computer and Information Science Bridges and Mediation in Higher Distance Education*. Laura Sara Agrati, Daniel Burgos, Pietro Ducange, Pierpaolo Limone, Loredana Perla, Pietro Picerno, Paolo Raviolo, and Christian M. Stracke (Eds.), Vol. 1344, Springer International Publishing, Cham, 243–255. DOI : https://doi.org/10.1007/978-3-030-67435-9_19
- [23] Massimo Aria and Corrado Cuccurullo. 2017. Bibliometrix: An R-tool for comprehensive science mapping analysis. *Journal of Informetrics* 11, 4 (2017), 959–975. DOI : <https://doi.org/10.1016/j.joi.2017.08.007>
- [24] William F. Atchison, Samuel D. Conte, John W. Hamblen, Thomas E. Hull, Thomas A. Keenan, William B. Kehl, Edward J. McCluskey, Silvio O. Navarro, Werner C. Rheinboldt, Earl J. Schweppe, et al. 1968. Curriculum 68: Recommendations for academic programs in computer science: A report of the ACM curriculum committee on computer science. *Communications of the ACM* 11, 3 (1968), 151–197.
- [25] Richard H. Austing, Bruce H. Barnes, Delle T. Bonnette, Gerald L. Engel, and Gordon Stokes. 1977. Curriculum recommendations for the undergraduate program in computer science: A working report of the ACM committee on curriculum in computer sciences. *ACM SIGCSE Bulletin* 9, 2 (1977), 1–16.
- [26] Thomas Baar. 2012. Improving software engineering education by modeling real-world implementations. In *Proceedings of the 8th Edition of the Educators' Symposium (EduSymp '12)*. ACM, New York, NY, 36–39. DOI : <https://doi.org/10.1145/2425936.2425943>
- [27] Jeroen Baas, Michiel Schotten, Andrew Plume, Grégoire Côté, and Reza Karimi. 2020. Scopus as a curated, high-quality bibliometric data source for academic research in quantitative science studies. *Quantitative Science Studies* 1, 1 (2020), 377–386. DOI : https://doi.org/10.1162/qss_a_00019

- [28] Catherine M. Baker, Yasmine N. El-Glaly, and Kristen Shinohara. 2020. A systematic analysis of accessibility in computing education research. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*, 107–113.
- [29] Christine Bakke and Rena Sakai. 2022. Using design-based research to layer career-like experiences onto software development courses. *Journal of Information Technology Education: Innovations in Practice* 21 (2022), 25–60. DOI : <https://doi.org/10.28945/4988>
- [30] Mathieu Bastian, Sebastien Heymann, and Mathieu Jacomy. 2009. Gephi: An open source software for exploring and manipulating networks. In *Proceedings of the 3rd International AAAI Conference on Weblogs and Social Media*. AAAI Press, Menlo Park, CA, 361–362.
- [31] Brett A. Becker and Keith Quille. 2019. 50 Years of CS1 at SIGCSE: A review of the evolution of introductory programming education research. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, 338–344.
- [32] Byron Weber Becker. 2001. Teaching CS1 with Karel the robot in Java. In *Proceedings of the 32nd SIGCSE Technical Symposium on Computer Science Education (SIGCSE '01)*. ACM, New York, NY, 50–54. DOI : <https://doi.org/10.1145/364447.364536>
- [33] Judith Bell and Tim Bell. 2018. Integrating computational thinking with a music education context. *Informatics in Education* 17, 2 (2018), 151–166.
- [34] Tim Bell, Frances Rosamond, and Nancy Casey. 2012. *Computer science unplugged and related projects in math and computer science popularization*. Springer Berlin, Berlin, 398–456. DOI : https://doi.org/10.1007/978-3-642-30891-8_18
- [35] Susan Bergin, Ronan Reilly, and Desmond Traynor. 2005. Examining the role of self-regulated learning on introductory programming performance. In *Proceedings of the 1st International Workshop on Computing Education Research (ICER '05)*. ACM, New York, NY, 81–86. DOI : <https://doi.org/10.1145/1089786.1089794>
- [36] João Henrique Berssanette and Antonio Carlos de Francisco. 2021. Active learning in the context of the teaching/learning of computer programming: A systematic review. *Journal of Information Technology Education: Research* 20 (2021), 201.
- [37] Briana Bettin, Linda Ott, and Julia Hiebel. 2022. Semaphore or metaphor? Exploring concurrent students' conceptions of and with analogy. In *Proceedings of the 27th ACM Conference on Innovation and Technology in Computer Science Education Vol. 1 (ITiCSE '22)*. ACM, New York, NY, 200–206. DOI : <https://doi.org/10.1145/3502718.3524796>
- [38] John B. Biggs and Kevin F. Collis. 2014. *Evaluating the Quality of Learning: The SOLO Taxonomy (Structure of the Observed Learning Outcome)*. Academic Press.
- [39] Javier Blanco, Leticia Losano, Nazareno Aguirre, María Marta Novaira, Sonia Permigiani, and Gastón Scilingo. 2009. An introductory course on programming based on formal specification and program calculation. *ACM SIGCSE Bulletin* 41, 2 (2009), 31–37.
- [40] Lenore Blum and Thomas J. Cortina. 2007. CS4HS: An outreach program for high school CS teachers. *ACM SIGCSE Bulletin* 39, 1 (2007), 19–23.
- [41] Nacir Bouali, Eeva Nygren, Solomon Sunday Oyelere, Jarkko Suhonen, and Violetta Cavalli-Sforza. 2019. Imikode: A VR game to introduce OOP concepts. In *Proceedings of the 19th Koli Calling International Conference on Computing Education Research (Koli Calling '19)*. ACM, New York, NY, Article 27, 1–2. DOI : <https://doi.org/10.1145/3364510.3366149>
- [42] Timothy A. Budd and Rajeev K. Pandey. 1995. Never mind *the* paradigm, what about multiparadigm languages? *ACM SIGCSE Bulletin* 27, 2 (1995), 25–30. DOI : <https://doi.org/10.1145/201998.202006>
- [43] Peter A. Buhr. 1985. A case for teaching multi-exit loops to beginning programmers. *ACM SIGPLAN Notices* 20, 11 (1985), 14–22.
- [44] Ing Carsten Busch, Regina Frieß, Martin Steinicke, and Til Wendler. 2021. Studying the impact of gamification on motivation in remote programming education. In *European Conference on e-Learning, Academic Conferences International Limited, Reading*, 627–634.
- [45] David Calleele and Dwight Makaroff. 2006. Teaching requirements engineering to an unsuspecting audience. In *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '06)*. ACM, New York, NY, 433–437. DOI : <https://doi.org/10.1145/1121341.1121475>.
- [46] Jennifer Campbell, Stan Kurkovsky, Chun Wai Liew, and Anya Tafilovich. 2016. Scrum and agile methods in software engineering courses. In *Proceedings of the 47th ACM Technical Symposium on Computing Science Education (SIGCSE '16)*. ACM, New York, NY, 319–320. DOI : <https://doi.org/10.1145/2839509.2844664>
- [47] Michal Černanský. 2023. Teaching synchronization in parallel computing. In *2023 21st International Conference on Emerging eLearning Technologies and Applications (ICETA)*. IEEE, 99–104.
- [48] Kuljit Kaur Chahal, Amanpreet Kaur, and Munish Saini. 2021. Empirical studies on using pair programming as a pedagogical tool in higher education courses: A systematic literature review. In *Research and Evidence in Software Engineering* (2021). Varun Gupta and Chetna Gupta (Eds.), Auerbach Publications, 251–286.

- [49] Manuel M. T. Chakravarty and Gabriele Keller. 2004. The risks and benefits of teaching purely functional programming in first year. *Journal of Functional Programming* 14, 1 (2004), 113–123.
- [50] Yi-Chun Chang, Jian-Wei Li, and De-Yao Huang. 2022. A personalized learning service compatible with moodle E-Learning management system. *Applied Sciences* 12, 7 (2022), 3562. DOI: <https://doi.org/10.3390/app12073562>
- [51] Guanhua Chen, Ji Shen, Lauren Barth-Cohen, Shiyang Jiang, Xiaoting Huang, and Moataz Eltoukhy. 2017. Assessing elementary students’ computational thinking in everyday reasoning and robotics programming. *Computers and Education* 109 (2017), 162–175. DOI: <https://doi.org/10.1016/j.compedu.2017.03.001>
- [52] Michael J. Clancy and Marcia C. Linn. 1990. Functional fun. In *Proceedings of the 21st SIGCSE Technical Symposium on Computer Science Education*, 63–67.
- [53] Alexandre Costa, Felipe Ramos, Mirko Perkusich, Emanuel Dantas, Ednaldo Dilenzo, Ferdinandy Chagas, André Meireles, Danylo Albuquerque, Luiz Silva, Hyggo Almeida, et al. 2020. Team formation in software engineering: A systematic mapping study. *IEEE Access* 8 (2020), 145687–145712.
- [54] Mayela Coto, Sonia Mora, Beatriz Grass, and Juan Murillo-Morera. 2022. Emotions and programming learning: Systematic mapping. *Computer Science Education* 32, 1 (2022), 30–65.
- [55] Tyne Crow, Andrew Luxton-Reilly, and Burkhard Wuensche. 2018. Intelligent tutoring systems for programming education: A systematic review. In *Proceedings of the 20th Australasian Computing Education Conference (ACE ’18)*. ACM, New York, NY, 53–62. DOI: <https://doi.org/10.1145/3160489.3160492>
- [56] José C. Cunha and João Lourenço. 1998. An integrated course on parallel and distributed processing. *ACM SIGCSE Bulletin* 30, 1 (1998), 217–221. DOI: <https://doi.org/10.1145/274790.274300>
- [57] Betul C. Czerkawski and Eugene W. Lyman. 2015. Exploring issues about computational thinking in higher education. *TechTrends* 59, 2 (March 2015), 57–65. DOI: <https://doi.org/10.1007/s11528-015-0840-3>
- [58] Valentina Dagienė, Yasemin Gülbahar, Natasa Grgurina, Sonsoles López-Pernas, Mohammed Saqr, Mikko Apiola, and Gabrielė Stupurienė. 2023. *Computing education research in schools*. In *Past, Present and Future of Computing Education Research: A Global Perspective*. Mikko Apiola, Sonsoles López-Pernas, and Mohammed Saqr (Eds.), Springer, Cham, 481–520. DOI: https://doi.org/10.1007/978-3-031-25336-2_20
- [59] Valentina Dagienė and Gabriele Stupurienė. 2016. Bebras—A sustainable community building model for the concept based learning of informatics and computational thinking. *Informatics in Education* 15, 1 (2016), 25–44.
- [60] Liam Davis-Wallace and Wu-Chun Feng. 2022. Parallel programming with pictures—A second path (work-in-progress). In *2022 IEEE Integrated STEM Education Conference (ISEC)*. IEEE. DOI: <https://doi.org/10.1109/isec54952.2022.10025156>
- [61] Pasquale De Meo, Emilio Ferrara, Giacomo Fiumara, and Alessandro Provetti. 2011. Generalized Louvain method for community detection in large networks. In *2011 11th International Conference on Intelligent Systems Design and Applications*. IEEE, 88–93. DOI: <https://doi.org/10.1109/ISDA.2011.6121636>
- [62] Constantinos T. Delistavrou and Konstantinos G. Margaritis. 2010. Survey of software environments for parallel distributed processing: Parallel programming education on real life target systems using production oriented software tools. In *2010 14th Panhellenic Conference on Informatics*. IEEE, 231–236. DOI: <https://doi.org/10.1109/PCI.2010.26>
- [63] Peter J. Denning. 2017. Remaining trouble spots with computational thinking. *Communications of the ACM* 60, 6 (May 2017), 33–39. DOI: <https://doi.org/10.1145/2998438>
- [64] Peter J. Denning and Matti Tedre. 2021. Computational thinking for professionals. *Communications of the ACM* 64, 12 (November 2021), 30–33. DOI: <https://doi.org/10.1145/3491268>
- [65] Brandon Dennis, Rita Kuo, and Ramyaa Ramyaa. 2023. Implementing interactive programming tutorials in object-oriented programming education. In *2023 IEEE International Conference on Advanced Learning Technologies (ICALT)*. IEEE, 180–181. DOI: <https://doi.org/10.1109/ICALT58122.2023.00058>
- [66] Paul Denny, Brett A. Becker, Nigel Bosch, James Prather, Brent Reeves, and Jacqueline Whalley. 2022. Novice reflections during the transition to a new programming language. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education—Volume 1 (SIGCSE ’22)*, ACM, New York, NY, 948–954. DOI: <https://doi.org/10.1145/3478431.3499314>
- [67] Vladan Devedžić and Saša R. Milenković. 2011. Teaching agile software development: A case study. *IEEE Transactions on Education* 54, 2 (2011), 273–278. DOI: <https://doi.org/10.1109/TE.2010.2052104>
- [68] Daghan Dinç and Suzan Üsküdarlı. 2009. A web environment to support teaching introductory programming. In *2009 4th International Conference on Internet and Web Applications and Services*. IEEE, 578–582. DOI: <https://doi.org/10.1109/ICIW.2009.93>
- [69] J. L. Donaldson, A. M. Lancaster, and P. H. Sposato. 1981. A plagiarism detection system. In *Proceedings of the 12th SIGCSE Technical Symposium on Computer Science Education*. Retrieved from <https://dl.acm.org/doi/abs/10.1145/800037.800955>

- [70] Wendy Doube. 2000. The impact on student performance of a change of language in successive introductory computer programming subjects. In *Proceedings of the Australasian Conference on Computing Education (ACSE '00)*. ACM, New York, NY, 71–78. DOI: <https://doi.org/10.1145/359369.359380>
- [71] Alessandra C. S. Dutra, Rafael Prikladnicki, and César França. 2015. What do we know about high performance teams in software engineering? Results from a systematic literature review. In *2015 41st Euromicro Conference on Software Engineering and Advanced Applications*. IEEE, 183–190.
- [72] Anna Eckerdal and Michael Thuné. 2005. Novice Java programmers' conceptions of "object" and "class", and variation theory. *ACM SIGCSE Bulletin* 37, 3 (2005), 89–93. DOI: <https://doi.org/10.1145/1151954.1067473>
- [73] Stephen H. Edwards. 2003. Improving student performance by evaluating how well students test their own programs. *Journal on Educational Resources in Computing* 3, 3 (2003), 1–es. DOI: <https://doi.org/10.1145/1029994.1029995>
- [74] Barbara J. Ericson, Paul Denny, James Prather, Rodrigo Duran, Arto Hellas, Juho Leinonen, Craig S. Miller, Briana B. Morrison, Janice L. Pearce, and Susan H. Rodger. 2022. Parsons problems and beyond: Systematic literature review and empirical study designs. In *Proceedings of the 2022 Working Group Reports on Innovation and Technology in Computer Science Education*, 191–234.
- [75] Janne Fagerlund, Päivi Häkkinen, Mikko Vesisenaho, and Jouni Viiri. 2021. Computational thinking in programming with scratch in primary schools: A systematic review. *Computer Applications in Engineering Education* 29, 1 (2021), 12–28. DOI: <https://doi.org/10.1002/cae.22255>
- [76] Katrina Falkner and Judy Sheard. 2019. 15 Pedagogic approaches. In *The Cambridge Handbook of Computing Education Research*. Sally A. Fincher and Anthony V. Robins (Eds.), Cambridge University Press, 445–480.
- [77] W. Feurzeig, S. Papert, M. Bloom, R. Grant, and C. Solomon. 1969. Programming-languages as a conceptual framework for teaching mathematics. Final Report on the First Fifteen Months of the LOGO Project.
- [78] Michal Forišek. 2010. The difficulty of programming contests increases. In *International Conference on Informatics in Secondary Schools-Evolution and Perspectives*. Springer, 72–85.
- [79] Mohammed Aristide Foughali. 2024. Some thoughts on teaching introductory programming and the first language dilemma (discussion paper). In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. ACM, New York, NY, Article 26, 1–8. DOI: <https://doi.org/10.1145/3631802.3631812>
- [80] Christiane Frede and Maria Knobelsdorf. 2021. A differentiated picture of student performance in introductory courses to theory of computation. *Computer Science Education* 31, 3 (2021), 315–339.
- [81] Frank L. Friedman and Elliot B. Koffman. 1976. Some pedagogic considerations in teaching elementary programming using structured FORTRAN. *ACM SIGCUE Outlook* 10, SI (1976), 1–10.
- [82] Ilenia Fronza, Arto Hellas, Petri Ihantola, and Tommi Mikkonen. 2020. Code reviews, software inspections, and code walkthroughs: Systematic mapping study of research topics. In *Proceedings of the 12th International Conference on Software Quality: Quality Intelligence in Software and Systems Engineering (SWQD '20)*. Springer, 121–133.
- [83] Anna Förster and Mehdi Jazayeri. 2010. Hands-on approach to teaching wireless sensor networks at the undergraduate level. In *Proceedings of the 15th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '10)*. ACM, New York, NY, 284–288. DOI: <https://doi.org/10.1145/1822090.1822170>
- [84] Viktor Geller and Christelle Scharff. 2005. Traditional and more "exotic".NET languages: VB.NET, J#, C# and SML.NET. In *Proceedings of the 10th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE '05)*. ACM, New York, NY, 406. DOI: <https://doi.org/10.1145/1067445.1067610>
- [85] Katerina Georgouli and Pedro Guerreiro. 2011. Integrating an automatic judge into an open source LMS. *International Journal on E-Learning* 10, 1 (2011), 27–42. Retrieved from <http://www.editlib.org/p/33289>
- [86] Norman E. Gibbs and Allen B. Tucker. 1986. A model curriculum for a liberal arts degree in computer science. *Communications of the ACM* 29, 3 (1986), 202–210.
- [87] Luis Mendes Gomes, Francisco Martins, and Hélia Guerra. 2020. Teaching web programming using the MEAN stack. In *Advances in Intelligent Systems and Computing*. Springer International Publishing, Cham, 256–262. DOI: https://doi.org/10.1007/978-3-030-40271-6_26
- [88] Tyler Greer, Qiang Hao, Mengguo Jing, and Bradley Barnes. 2019. On the effects of active learning environments in computing education. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19)*. ACM, New York, NY, 267–272. DOI: <https://doi.org/10.1145/3287324.3287345>
- [89] Shuchi Grover and Roy Pea. 2013. Computational thinking in K–12: A review of the state of the field. *Educational Researcher* 42, 1 (2013), 38–43. DOI: <https://doi.org/10.3102/0013189X12463051>
- [90] Shuchi Grover, Roy Pea, and Stephen Cooper. 2015. Designing for deeper learning in a blended computer science course for middle school students. *Computer Science Education* 25, 2 (2015), 199–237. DOI: <https://doi.org/10.1080/08993408.2015.1033142>
- [91] Mark Guzdial and Benedict Du Boulay. 2019. The history of computing. In *The Cambridge Handbook of Computing Education Research*. Sally A. Fincher and Anthony V. Robins (Eds.), Cambridge University Press, 11–39.

- [92] Brian Hanks, Sue Fitzgerald, Renée McCauley, Laurie Murphy, and Carol Zander. 2011. Pair programming in education: A literature review. *Computer Science Education* 21, 2 (2011), 135–173.
- [93] Anja Hawlitschek, Sarah Berndt, and Sandra Schulz. 2023. Empirical research on pair programming in higher education: A literature review. *Computer Science Education* 33, 3 (2023), 400–428.
- [94] Arto Hellas, Petri Ihantola, Andrew Petersen, Vangel V. Ajanovski, Mirela Gutica, Timo Hynninen, Antti Knutas, Juho Leinonen, Chris Messom, and Soohyun Nam Liao. 2018. Taxonomizing features and methods for identifying at-risk students in computing courses. In *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*, 364–365.
- [95] Peter B. Henderson and Francisco J. Romero. 1989. Teaching recursion as a problem-solving tool using standard ML. *ACM SIGCSE Bulletin* 21, 1 (1989), 27–31.
- [96] Hendrik Hendrik and Almed Hamzah. 2021. Flipped classroom in programming course: A systematic literature review. *International Journal of Emerging Technologies in Learning* 16, 2 (2021), 220–236.
- [97] Paul N. Hilfinger, Mary Shaw, and Wm. A. Wulf. 1978. Introducing “theory” in the second programming course. In *Proceedings of the 9th SIGCSE Technical Symposium on Computer Science Education*, 55–58.
- [98] Makoto J. Hirayama and Toshiyuki Yamamoto. 2011. Web programming education through developing online shop web application. In *Edutainment Technologies. Educational Games and Virtual Reality/Augmented Reality Applications*. Springer Berlin, Berlin, 397–398. DOI: https://doi.org/10.1007/978-3-642-23456-9_69
- [99] Amanda Holland-Minkley, Jakob Barnard, Valerie Barr, Grant Braught, Janet Davis, David Reed, Karl Schmitt, Andrea Tartaro, and James D. Teresco. 2023. Computer science curriculum guidelines: A new liberal arts perspective. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education*, 617–623.
- [100] Helen H. Hu, Aman Yadav, Donna M. Gavin, Clif Kussmaul, and Chris Mayfield. 2023. Teamwork in CS1: Student learning and experience with POGIL. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education*, 729–735.
- [101] Joe Hummel and Jean Mehta. 2002. Using visual basic in the CS curriculum. *ACM SIGCSE Bulletin* 34, 1 (2002), 283–284. DOI: <https://doi.org/10.1145/563517.563451>
- [102] Corinna Hörmann, Marina Rottenhofer, Iris Groher, and Barbara Sabitzer. 2021. Let the games begin - Inviting young learners to code. In *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '21)*. ACM, New York, NY, 644. DOI: <https://doi.org/10.1145/3456565.3460074>
- [103] Mohamud Sheikh Ibrahim, Norsaremah Salleh, and Sanjay Misra. 2015. Empirical studies of cloud computing in education: A systematic literature review. In *Proceedings of the 15th International Conference on Computational Science and Its Applications (ICCSA '15)*, Part IV. Springer, 725–737.
- [104] Petri Ihantola, Tuukka Ahoniemi, Ville Karavirta, and Otto Seppälä. 2010. Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research (Koli Calling '10)*. ACM, New York, NY, 86–93. DOI: <https://doi.org/10.1145/1930464.1930480>
- [105] Petri Ihantola, Arto Vihavainen, Alireza Ahadi, Matthew Butler, Jürgen Börstler, Stephen H. Edwards, Essi Isohanni, Ari Korhonen, Andrew Petersen, Kelly Rivers, et al. 2015. Educational data mining and learning analytics in programming: Literature review and case studies. In *Proceedings of the 2015 ITiCSE on Working Group Reports*, 41–63.
- [106] Ambikesh Jayal, Stasha Lauria, Allan Tucker, and Stephen Swift. 2011. Python for teaching introductory programming: A quantitative evaluation. *Innovation in Teaching and Learning in Information and Computer Sciences* 10, 1 (2011), 86–90. DOI: <https://doi.org/10.11120/ital.2011.10010086>
- [107] G. Jayathirtha. [n. d.] Computational concepts, practices, and collaboration in high school students’ debugging electronic textile projects. In *Conference Proceedings of International Conference on Computational Thinking Education 2018*. Retrieved from <https://par.nsf.gov/biblio/10061523>
- [108] Yasmin B. Kafai. 2016. From computational thinking to computational participation in K–12 education. *Communications of the ACM* 59, 8 (July 2016), 26–27. DOI: <https://doi.org/10.1145/2955114>
- [109] Yasmin B. Kafai and Quinn Burke. 2013. The social turn in K-12 programming: Moving from computational thinking to computational participation. In *Proceeding of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE '13)*. ACM, New York, NY, 603–608. DOI: <https://doi.org/10.1145/2445196.2445373>
- [110] Yasmin B. Kafai, Deborah A. Fields, Debora A. Lui, Justice T. Walker, Mia S. Shaw, Gayithri Jayathirtha, Tomoko M. Nakajima, Joanna Goode, and Michael T. Giang. 2019. Stitching the loop with electronic textiles: Promoting equity in high school students’ competencies and perceptions of computer science. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, 1176–1182.
- [111] Rogers Kaliisa, Sonsoles López-Pernas, Kamila Misiejuk, Crina Damşa, Márta Sobocinski, Sanna Järvelä, and Mohammed Saqr. 2025. A topical review of research in computer-supported collaborative learning: Questions and possibilities. *Computers and Education* 228 (2025), 105246.

- [112] Fuat Keceli, Alexandros Tzannes, George C. Caragea, Rajeev Barua, and Uzi Vishkin. 2011. Toolchain for programming, simulating and studying the XMT many-core architecture. In *2011 IEEE International Symposium on Parallel and Distributed Processing Workshops and PhD Forum*. IEEE, 1282–1291. DOI : <https://doi.org/10.1109/IPDPS.2011.270>
- [113] Caitlin Kelleher and Randy Pausch. 2005. Lowering the barriers to programming: A taxonomy of programming environments and languages for novice programmers. *ACM Computing Surveys* 37, 2 (June 2005), 83–137. DOI : <https://doi.org/10.1145/1089733.1089734>
- [114] Caitlin Kelleher, Randy Pausch, and Sara Kiesler. 2007. Storytelling Alice motivates middle school girls to learn computer programming. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, 1455–1464. DOI : <https://doi.org/10.1145/1240624.1240844>
- [115] Riko Kelter, Matthias Kramer, and Torsten Brinda. 2018. Statistical frequency-analysis of misconceptions in object-oriented-programming: Regularized PCR models for frequency analysis across OOP concepts and related factors. In *Proceedings of the 18th Koli Calling International Conference on Computing Education Research (Koli Calling '18)*. ACM, New York, NY, Article 6, 1–10. DOI : <https://doi.org/10.1145/3279720.3279727>
- [116] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2016. Towards a systematic review of automated feedback generation for programming exercises. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '16)*. ACM, New York, NY, 41–46. DOI : <https://doi.org/10.1145/2899415.2899422>
- [117] Jeong Ah Kim and Dae Young Ko. 2018. Survey of on-line and block programming language - Scratch: On perspective of educational achievements. In *Advances in Computer Science and Ubiquitous Computing*. Springer Singapore, 35–40. DOI : https://doi.org/10.1007/978-981-10-7605-3_6
- [118] Gerd Kortuem, Arosha K. Bandara, Neil Smith, Mike Richards, and Marian Petre. 2013. Educating the internet-of-things generation. *Computer* 46, 2 (2013), 53–61.
- [119] David R. Krathwohl. 2002. A revision of bloom's taxonomy: An overview. *Theory Into Practice* 41, 4 (2002), 212–218.
- [120] Stan Kurkovsky. 2009. Engaging students through mobile game development. In *Proceedings of the 40th ACM Technical Symposium on Computer Science Education (SIGCSE '09)*. ACM, New York, NY, 44–48. DOI : <https://doi.org/10.1145/1508865.1508881>
- [121] C. Peter Lawes. 1974. Description of an introductory course in programming languages. *ACM SIGCSE Bulletin* 6, 3 (1974), 69–78.
- [122] Arthur H. Lee. 2003. A manageable web software architecture: Searching for simplicity. *ACM SIGCSE Bulletin* 35, 1 (2003), 229–233. DOI : <https://doi.org/10.1145/792548.611976>
- [123] R. R. Leeper and J. L. Silver. 1982. Predicting success in a first programming course. *ACM SIGCSE Bulletin* 14, 1 (1982), 147–150. DOI : <https://doi.org/10.1145/953051.801357>
- [124] Scott Leutenegger and Jeffrey Edgington. 2007. A games first approach to teaching introductory programming. In *Proceedings of the 38th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '07)*. ACM, New York, NY, 115–118. DOI : <https://doi.org/10.1145/1227310.1227352>
- [125] Alex Lishinski, Jon Good, Phil Sands, and Aman Yadav. 2016. Methodological rigor and theoretical foundations of CS education research. In *12th International Computing Education Research Conference (ICER '16)*, 161–169.
- [126] Raymond Lister. 2011. Concrete and other neo-Piagetian forms of reasoning in the novice programmer. In *Proceedings of the Thirteenth Australasian Computing Education Conference (ACE 2011)*, Perth, Australia. Conferences in Research and Practice in Information Technology Series, Vol. 114, 9–19.
- [127] Raymond Lister, Elizabeth S. Adams, Sue Fitzgerald, William Fone, John Hamer, Morten Lindholm, Robert McCartney, Jan Erik Moström, Kate Sanders, Otto Seppälä, et al. 2004. A multi-national study of reading and tracing skills in novice programmers. *ACM SIGCSE Bulletin* 36, 4 (2004), 119–150.
- [128] Raymond Lister, Anders Berglund, Tony Clear, Joe Bergin, Kathy Garvin-Doxas, Brian Hanks, Lew Hitchner, Andrew Luxton-Reilly, Kate Sanders, Carsten Schulte, et al. 2006. Research perspectives on the objects-early debate. *ACM SIGCSE Bulletin* 38, 4 (2006), 146–165.
- [129] Yan Liu. 2015. Teaching programming on cloud: A perspective beyond programming. In *IEEE 7th International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 594–599. DOI : <https://doi.org/10.1109/CloudCom.2015.101>
- [130] Yanhong A. Liu and Matthew Castellana. 2021. Discrete math with programming: A principled approach. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*, 1156–1162.
- [131] Michael Lodi, Dario Malchiodi, Mattia Monga, Anna Morpurgo, and Bernadette Spieler. 2019. Constructionist attempts at supporting the learning of computer programming: A survey. *Olympiads in Informatics* 13 (2019), 99–121.
- [132] Sonsoles López-Pernas, Mikko Apiola, Mohammed Saqr, Arnold Pears, and Matti Tedre. 2023. A scientometric perspective on the evolution of the SIGCSE technical symposium: 1970–2021. In *Past, Present and Future of Computing Education Research: A Global Perspective*. Mikko Apiola, Sonsoles López-Pernas, and Mohammed Saqr (Eds.), Springer, 193–212.

- [133] Sonsoles López-Pernas, Mohammed Saqr, and Mikko Apiola. 2023. Scientometrics: A concise introduction and a detailed methodology for the mapping of the scientific field of computing education. In *Past, Present and Future of Computing Education Research*. Mikko Apiola, Sonsoles López-Pernas, and Mohammed Saqr (Eds.), Springer.
- [134] Sonsoles López-Pernas, Mohammed Saqr, Miguel Á. Conde, Mikko Apiola, and Matti Tedre. 2025. Mapping computer engineering education research: A topic analysis. *Computer* 58, 1 (2025), 66–75.
- [135] Aleksi Lukkarinen, Lauri Malmi, and Lassi Haaranen. 2021. Event-driven programming in programming education: A mapping review. *ACM Transactions on Computing Education (TOCE)* 21, 1 (2021), 1–31.
- [136] Andrew Luxton-Reilly, Simon, Ibrahim Alblawi, Brett A. Becker, Michail Giannakos, Amruth N. Kumar, Linda Ott, James Paterson, Michael James Scott, Judy Sheard, and Claudia Szabo. 2018. Introductory programming: A systematic literature review. In *ITiCSE 2018 Working Group Reports (ITiCSE WGR '18)*, 55–106.
- [137] Sze Yee Lye and Joyce Hwee Ling Koh. 2014. Review on teaching and learning of computational thinking through programming: What is next for K-12? *Computers in Human Behavior* 41 (2014), 51–61. DOI: <https://doi.org/10.1016/j.chb.2014.09.012>
- [138] Sonsoles López-Pernas, Aldo Gordillo, Enrique Barra, and Juan Quemada. 2019. Examining the use of an educational escape room for teaching programming in a higher education setting. *IEEE Access* 7 (2019), 31723–31737. DOI: <https://doi.org/10.1109/ACCESS.2019.2902976>
- [139] Mary Lou Maher, Celine Latulipe, Heather Lipford, and Audrey Rorrer. 2015. Flipped classroom strategies for CS education. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education (SIGCSE '15)*. ACM, New York, NY, 218–223. DOI: <https://doi.org/10.1145/2676723.2677252>
- [140] Louis Major, Theocharis Kyriacou, and O. Pearl Brereton. 2012. Systematic literature review: Teaching novices programming using robots. *IET Software* 6, 6 (2012), 502–513.
- [141] Lauri Malmi, Judy Sheard, Päivi Kinnunen, Simon, and Jane Sinclair. 2019. Computing education theories: What are they and how are they used? In *15th International Computing Education Research Conference (ICER '19)*, 187–197.
- [142] Lauri Malmi, Judy Sheard, Päivi Kinnunen, Simon, and Jane Sinclair. 2020. Theories and models of emotions, attitudes, and self-efficacy in the context of programming education. In *16th International Computing Education Research Conference (ICER '20)*, 36–47.
- [143] Lauri Malmi, Judy Sheard, Simon, Roman Bednarik, Juha Helminen, Päivi Kinnunen, Ari Korhonen, Niko Myller, Juha Sorva, and Ahmad Taherkhani. 2014. Theoretical underpinnings of computing education research: What is the evidence? In *10th International Computing Education Research Conference (ICER '14)*, 27–34.
- [144] Lauri Malmi, Jane Sinclair, Judy Sheard, Simon, and Päivi Kinnunen. 2023. *The Evolution of Computing Education Research: A Meta-Analytic Perspective*. In *Past, Present and Future of Computing Education Research: A Global Perspective*. Mikko Apiola, Sonsoles López-Pernas, and Mohammed Saqr (Eds.), Springer, Cham, 51–77. DOI: https://doi.org/10.1007/978-3-031-25336-2_4
- [145] Lauri Malmi, Ian Utting, and Andrew J. Ko. 2019. Tools and environments. In *The Cambridge Handbook of Computing Education Research*. Sally A. Fincher and Anthony V. Robins (Eds.), Cambridge University Press, 639–662.
- [146] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. 2010. The Scratch programming language and environment. *ACM Transactions on Computing Education* 10, 4 (November 2010), Article 16, 1–15. DOI: <https://doi.org/10.1145/1868358.1868363>
- [147] John H. Maloney, Kylie Peppler, Yasmin Kafai, Mitchel Resnick, and Natalie Rusk. 2008. Programming by choice: Urban youth learning programming with Scratch. In *Proceedings of the 39th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '08)*. ACM, New York, NY, 367–371. DOI: <https://doi.org/10.1145/1352135.1352260>
- [148] Lauren E. Margulieux and Briana B. Morrison. 2019. Guest editorial. *Computer Science Education* 29, 2-3 (2019), 103–105.
- [149] Lauren E. Margulieux, Briana B. Morrison, and Adrienne Decker. 2020. Reducing withdrawal and failure rates in introductory programming with subgoal labeled worked examples. *International Journal of STEM Education* 7 (2020), 1–16.
- [150] Robert Marmorstein. 2011. Open source contribution as an effective software engineering class project. In *Proceedings of the 16th Annual Joint Conference on Innovation and Technology in Computer Science Education (ITiCSE '11)*. ACM, New York, NY, 268–272. DOI: <https://doi.org/10.1145/1999747.1999823>
- [151] Renee McCauley, Sue Fitzgerald, Gary Lewandowski, Laurie Murphy, Beth Simon, Lynda Thomas, and Carol Zander. 2008. Debugging: A review of the literature from an educational perspective. *Computer Science Education* 18, 2 (2008), 67–92.
- [152] Michael McCracken, Vicki Almstrum, Danny Diaz, Mark Guzdial, Dianne Hagan, Yifat Ben-David Kolikant, Cary Laxer, Lynda Thomas, Ian Utting, and Tadeusz Wilusz. 2001. A multi-national, multi-institutional study of assessment of programming skills of first-year CS students. In *Working Group Reports from ITiCSE on Innovation and Technology in Computer Science Education*, 125–180.

- [153] Kirby McMaster, Nicole Anderson, and Brian Rague. 2007. Discrete math with programming: Better together. *ACM SIGCSE Bulletin* 39, 1 (2007), 100–104.
- [154] Orni Meerbaum-Salant, Michal Armoni, and Mordechai Ben-Ari. 2010. Learning computer science concepts with Scratch. In *Proceedings of the 6th International Workshop on Computing Education Research*, 69–76.
- [155] Pekka Mertala, Sonsoles López-Pernas, Henriikka Vartiainen, Mohammed Saqr, and Matti Tedre. 2024. Digital natives in the scientific literature: A topic modeling approach. *Computers in Human Behavior* 152 (2024), 108076.
- [156] Marcus Messer, Neil C. C. Brown, Michael Kölling, and Miaojing Shi. 2024. Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education* 24, 1 (2024), 1–43. DOI: <https://doi.org/10.1145/3636515>
- [157] Lance A. Miller. 1974. Programming by non-programmers. *International Journal of Man-Machine Studies* 6, 2 (1974), 237–260.
- [158] John Mingers and Loet Leydesdorff. 2015. A review of theory and practice in scientometrics. *European Journal of Operational Research* 246, 1 (2015), 1–19.
- [159] N. Myller, J. Suhonen, and E. Sutinen. 2002. Using data mining for improving web-based course design. In *Proceedings of the International Conference on Computers in Education*, 2002. IEEE, 959–963. DOI: <https://doi.org/10.1109/CIE.2002.1186125>
- [160] Mamoun Nawahdah and Majdi Jaradat. 2022. The effect of pair programming on code maintainability. In *Collaboration Technologies and Social Computing*. Lung-Hsiang Wong, Yugo Hayashi, Cesar A. Collazos, Claudio Alvarez, Gustavo Zurita, and Nelson Baloian (Eds.), CollabTech 2022. Lecture Notes in Computer Science, Vol. 13632, Springer International Publishing, Cham, 38–51. DOI: https://doi.org/10.1007/978-3-031-20218-6_3
- [161] Matti Nelimarkka and Arto Hellas. 2018. Social help-seeking strategies in a programming MOOC. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education (SIGCSE '18)*. ACM, New York, NY, 116–121. DOI: <https://doi.org/10.1145/3159450.3159495>
- [162] A. E. Nicholson and K. M. Fraser. 1997. Methodologies for teaching new programming languages: A case study teaching LISP. In *Proceedings of the 2nd Australasian Conference on Computer Science Education (ACSE '97)*. ACM, New York, NY, 84–90. DOI: <https://doi.org/10.1145/299359.299373>
- [163] Matija Novak, Mike Joy, and Dragutin Kermek. 2019. Source-code similarity detection and detection tools used in academia: A systematic review. *ACM Transactions on Computing Education* 19, 3 (2019), 1–37. DOI: <https://doi.org/10.1145/3313290>
- [164] José Carlos Paiva, José Paulo Leal, and Álvaro Figueira. 2022. Automated assessment in computer science education: A state-of-the-art review. *ACM Transactions on Computing Education* 22, 3 (2022), 1–40. DOI: <https://doi.org/10.1145/3513140>
- [165] Marina Papastergiou. 2008. Are computer science and information technology still masculine fields? High school students' perceptions and career choices. *Computers and Education* 51, 2 (2008), 594–608.
- [166] Seymour A. Papert. 1980. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books.
- [167] Arnold Pears, Stephen Seidman, Lauri Malmi, Linda Mannila, Elizabeth Adams, Jens Bennedsen, Marie Devlin, and James Paterson. 2007. A survey of literature on the teaching of introductory programming. In *Working Group Reports on ITiCSE on Innovation and Technology in Computer Science Education*, 204–223.
- [168] Piumi Perera, Geethya Tennakoon, Supunmali Ahangama, Rangana Panditharathna, and Buddhika Chathuranga. 2021. A systematic mapping of introductory programming languages for novice learners. *IEEE Access* 9 (2021), 88121–88136.
- [169] Chris Piech, Mehran Sahami, Daphne Koller, Steve Cooper, and Paulo Blikstein. 2012. Modeling how students learn to program. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE '12)*. ACM, New York, NY, 153–160. DOI: <https://doi.org/10.1145/2157136.2157182>
- [170] Lori Pollock, Kathleen McCoy, Sandra Carberry, Namratha Hundigopal, and Xiaoxin You. 2004. Increasing high school girls' self confidence and awareness of CS through a positive summer experience. *ACM SIGCSE Bulletin* 36, 1 (2004), 185–189.
- [171] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, et al. 2023. The robots are here: Navigating the generative ai revolution in computing education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, 108–159.
- [172] Marina Prvan and Julije Ožegović. 2020. Methods in teaching computer networks: A literature review. *ACM Transactions on Computing Education* 20, 3 (2020), 1–35.
- [173] John R. Pugh, Wilf R. LaLonde, and David A. Thomas. 1987. Introducing object-oriented programming into the computer science curriculum. In *Proceedings of the 18th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '87)*. ACM, New York, NY, 98–102. DOI: <https://doi.org/10.1145/31820.31742>

- [174] Kanjun Qiu, Leah Buechley, Edward Baafi, and Wendy Dubow. 2013. A curriculum for teaching computer science through computational textiles. In *Proceedings of the 12th International Conference on Interaction Design and Children (IDC '13)*. ACM, New York, NY, 20–27. DOI : <https://doi.org/10.1145/2485760.2485787>
- [175] Noa Ragonis and Mordechai Ben-Ari. 2005. A long-term investigation of the comprehension of OOP concepts by novices. *Computer Science Education* 15, 3 (2005), 203–221. DOI : <https://doi.org/10.1080/08993400500224310>
- [176] Anthony Ralston. 1971. Fortran and the first course in computer science. *ACM SIGCSE Bulletin* 3, 4 (1971), 24–29.
- [177] Molly Roberts, Brandon Stewart, Dustin Tingley, and Edoardo Airolidi. 2013. The structural topic model and applied social science. *Neural Information Processing Society*. 2013. Retrieved from <https://mimno.infosci.cornell.edu/nips2013ws/slides/stm.pdf>
- [178] Margaret E. Roberts, Brandon M. Stewart, and Dustin Tingley. 2019. Stm: An R package for structural topic models. *Journal of Statistical Software* 91 (2019), 1–40. DOI : <https://doi.org/10.18637/jss.v091.i02>
- [179] Margaret E. Roberts, Brandon M. Stewart, Dustin Tingley, Christopher Lucas, Jetson Leder-Luis, Shana Kushner Gadarian, Bethany Albertson, and David G. Rand. 2014. Structural topic models for open-ended survey responses. *American Journal of Political Science* 58, 4 (2014), 1064–1082.
- [180] Anthony Robins, Janet Rountree, and Nathan Rountree. 2003. Learning and teaching programming: A review and discussion. *Computer Science Education* 13, 2 (2003), 137–172.
- [181] Anthony V. Robins. 2019. Novice programmers and introductory programming. In *The Cambridge Handbook of Computing Education Research*. Sally A. Fincher and Anthony V. Robins (Eds.), Cambridge University Press, 327–376.
- [182] Daisuke Saito, Ayana Sasaki, Hironori Washizaki, Yoshiaki Fukazawa, and Yusuke Muto. 2017. Program learning for beginners: Survey and taxonomy of programming learning tools. In *2017 IEEE 9th International Conference on Engineering Education (ICEED)*. IEEE, 137–142. DOI : <https://doi.org/10.1109/ICEED.2017.8251181>
- [183] Simone C. Santos, Patricia Azevedo Tedesco, Matheus Borba, and Matheus Brito. 2020. Innovative approaches in teaching programming: A systematic literature review. In *Proceedings of the 12th International Conference on Computer Supported Education*, 205–214.
- [184] Mohammed Saqr, Kwok Ng, Solomon Sunday Oyelere, and Matti Tedre. 2021. People, ideas, milestones: A scientometric study of computational thinking. *ACM Transactions on Computing Education* 21, 3 (March 2021), Article 20, 1–17. DOI : <https://doi.org/10.1145/3445984>
- [185] Lilian Passos Scatalon, Jeffrey C. Carver, Rogério Eduardo Garcia, and Ellen Francine Barbosa. 2019. Software testing in introductory programming courses: A systematic mapping study. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, 421–427.
- [186] Alexandra Schofield and David Mimno. 2016. Comparing apples to apple: The effects of stemmers on topic models. *Transactions of the Association for Computational Linguistics* 4 (2016), 287–300. DOI : https://doi.org/10.1162/tac1_a_00099
- [187] Sue Sentance and Jane Waite. 2017. PRIMM: Exploring pedagogical approaches for teaching text-based programming in school. In *Proceedings of the 12th Workshop on Primary and Secondary Computing Education (WiPSCE '17)*. ACM, New York, NY, 113–114. DOI : <https://doi.org/10.1145/3137065.3137084>
- [188] Clifford A. Shaffer, Matthew L. Cooper, Alexander Joel D. Alon, Monika Akbar, Michael Stewart, Sean Ponce, and Stephen H. Edwards. 2010. Algorithm visualization: The state of the field. *ACM Transactions on Computing Education* 10, 3 (2010), 1–22. DOI : <https://doi.org/10.1145/1821996.1821997>
- [189] Sadi Sharmin. 2021. Creativity in CS1: A literature review. *ACM Transactions on Computing Education* 22, 2 (2021), 1–26.
- [190] Judy Sheard, Paul Denny, Arto Hellas, Juho Leinonen, and Lauri Malmi. 2024. Instructor perceptions of AI code generation tools – A multi-Institutional interview study. In *SIGCSE Technical Symposium*, 1223–1229.
- [191] Judy Sheard, S. Simon, Margaret Hamilton, and Jan Lönnberg. 2009. Analysis of research into the teaching and learning of programming. In *Proceedings of the 5th International Workshop on Computing Education Research Workshop*, 93–104.
- [192] Robert J. Sheehan. 2007. Teaching operating systems with Ruby. In *Proceedings of the 12th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE '07)*. ACM, New York, NY, 38–42. DOI : <https://doi.org/10.1145/1268784.1268798>
- [193] Esther Shein. 2015. Python for beginners. *Communications of the ACM* 58, 3 (2015), 19–21. DOI : <https://doi.org/10.1145/2716560>
- [194] Anastasia Shipepe, Ilkka Jormanainen, and Erkki Sutinen. 2020. Educational robotics initiatives in Namibia and worldwide. In *8th International Conference on Technological Ecosystems for Enhancing Multiculturality (TEEM'20)*. ACM, New York, NY, 48–53. DOI : <https://doi.org/10.1145/3434780.3436675>
- [195] Shuhaida Shuhidan, Margaret Hamilton, and Daryl D'souza. 2009. A taxonomic study of novice programming summative assessment. In *Proceedings of the 11th Australasian Conference on Computing Education*. Citeseer, 147–156.

- [196] V. J. Shute and S. Rahimi. 2017. Review of computer-based assessment for learning in elementary and secondary education. *Journal of Computer Assisted Learning* 33, 1 (2017), 1–19. DOI: <https://doi.org/10.1111/jcal.12172>
- [197] Josivan Silva and Ismar Silveira. 2020. A systematic review on open educational games for programming learning and teaching. *International Journal of Emerging Technologies in Learning* 15, 9 (2020), 156–172.
- [198] Leonardo Silva, António José Mendes, and Anabela Gomes. 2020. Computer-supported collaborative learning in programming education: A systematic literature review. In *2020 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, 1086–1095.
- [199] Simon, Andrew Luxton-Reilly, Vangel V. Ajanovski, Eric Fouh, Christabel Gonsalvez, Juho Leinonen, Jack Parkinson, Matthew Poole, and Neena Thota. 2019. Pass rates in introductory programming and in other STEM disciplines. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education*, 53–71.
- [200] Lyle B. Smith. 1967. A comparison of batch processing and instant turnaround. *Communications of the ACM* 10, 8 (1967), 495–500. DOI: <https://doi.org/10.1145/363534.363552>
- [201] Igor Solecki, João Porto, Nathalia da, Cruz Alves, Christiane Gresse von Wangenheim, Jean Hauck, and Adriano Ferreti Borgatto. 2020. Automated assessment of the visual design of Android apps developed with App Inventor. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE '20)*. ACM, New York, NY, 51–57. DOI: <https://doi.org/10.1145/3328778.3366868>
- [202] Cynthia Solomon, Brian Harvey, Ken Kahn, Henry Lieberman, Mark L. Miller, Margaret Minsky, Artemis Papert, and Brian Silverman. 2020. History of Logo. *Proceedings of the ACM on Programming Languages* 4, HOPL (June 2020), Article 79, 1–66. DOI: <https://doi.org/10.1145/3386329>
- [203] Juha Sorva, Ville Karavirta, and Lauri Malmi. 2013. A review of generic program visualization systems for introductory programming education. *ACM Transactions on Computing Education* 13, 4 (2013), 1–64.
- [204] Jaime Spacco and William Pugh. 2006. Helping students appreciate test-driven development (TDD). In *Companion to the 21st ACM SIGPLAN Symposium on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '06)*. ACM, New York, NY, 907–913. DOI: <https://doi.org/10.1145/1176617.1176743>
- [205] Andreas Stefik and Susanna Siebert. 2013. An empirical investigation into programming language syntax. *ACM Transactions on Computing Education* 13, 4 (2013), 1–40.
- [206] Milan Straka and Jana Straková. 2017. Tokenizing, POS tagging, lemmatizing and parsing UD 2.0 with UDPipe. In *Proceedings of the CoNLL 2017 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies*. Association for Computational Linguistics, Vancouver, Canada, 88–99. DOI: <https://doi.org/10.18653/v1/K17-3009>
- [207] Oleg Sukhoroslov. 2017. *A Service-Oriented Infrastructure for Teaching Big Data Technologies*. Springer International Publishing, Cham, 505–515. DOI: https://doi.org/10.1007/978-3-319-71255-0_41
- [208] Amanda Sullivan and Marina Umaschi Bers. 2019. Investigating the use of robotics to increase girls' interest in engineering during early elementary school. *International Journal of Technology and Design Education* 29, 5 (2019), 1033–1051.
- [209] Lihui Sun, Zhen Guo, and Danhua Zhou. 2022. Developing K-12 students' programming ability: A systematic literature review. *Education and Information Technologies* 27, 5 (2022), 7059–7097.
- [210] Thomas Suselo, Burkhard C. Wünsche, and Andrew Luxton-Reilly. 2019. Technologies and tools to support teaching and learning computer graphics: A literature review. In *Proceedings of the 21st Australasian Computing Education Conference*, 96–105.
- [211] Yan Watequlis Syaifudin, Nobuo Funabiki, Minoru Kuribayashi, and Wen-Chung Kao. 2019. A proposal of Android programming learning assistant system with implementation of basic application learning. *International Journal of Web Information Systems* 16, 1 (2019), 115–135. DOI: <https://doi.org/10.1108/ijwis-08-2019-0038>
- [212] Yan Watequlis Syaifudin, Nobuo Funabiki, Mustika Mentari, and Muhammad Sofiul Fuad. 2021. An implementation of multimedia resources learning topic for interactive applications in Android programming learning assistance system. In *2021 4th International Conference on Vocational Education and Electrical Engineering (ICVEE)*. IEEE, 1–5. DOI: <https://doi.org/10.1109/ICVEE54186.2021.9649696>
- [213] Claudia Szabo, Nickolas Falkner, Andrew Petersen, Heather Bort, Kathryn Cunningham, Peter Donaldson, Arto Hellas, James Robinson, and Judy Sheard. 2019. Review and use of learning theories within computer science education research: Primer for researchers and practitioners. In *ITiCSE 2019 Working Group Reports*, 89–109.
- [214] Claudia Szabo and Judy Sheard. 2023. Learning theories use and relationships in computing education research. *ACM Transactions on Computing Education* 23, 1 (March 2023), Article 5, 1–34. DOI: <https://doi.org/10.1145/3487056>
- [215] Wing C. Tam. 1992. Teaching loop invariants to beginners by examples. In *Proceedings of the 23rd SIGCSE Technical Symposium on Computer Science Education*, 92–96.
- [216] Matti Tedre, Peter Denning, and Tapani Toivonen. 2021. CT 2.0. In *21st Koli Calling International Conference on Computing Education Research (Koli Calling '21)*. ACM, New York, NY, Article 3, 1–8. DOI: <https://doi.org/10.1145/3488042.3488053>

- [217] Matti Tedre and Peter J. Denning. 2016. The long quest for computational thinking. In *Proceedings of the 16th Koli Calling International Conference on Computing Education Research (Koli Calling '16)*. ACM, New York, NY, 120–129. DOI: <https://doi.org/10.1145/2999541.2999542>
- [218] F. Boray Tek, Kristin S. Benli, and Ezgi Deveci. 2018. Implicit theories and self-efficacy in an introductory programming course. *IEEE Transactions on Education* 61, 3 (2018), 218–225.
- [219] Josh Tenenber, Sally Fincher, Ken Blaha, Dennis Bouvier, Tzu-Yi Chen, Donald Chinn, Stephen Cooper, Anna Eckerdal, Hubert Johnson, Robert McCartney, et al. 2005. Students designing software: A multi-national, multi-institutional study. *Informatics in Education* 4, 1 (2005), 143–162.
- [220] Josh Tenenber and Lauri Malmi. 2022. Conceptualizing and using theory in computing education research. *ACM Transactions on Computing Education* 22, 4 (2022), 1–8.
- [221] Richard M. Thall. 1983. Large-scale software development with the Ada language system. In *Proceedings of the 1983 Computer Science Conference (CSC '83)*. ACM, New York, NY, 55–67. DOI: <https://doi.org/10.1145/800172.809679>
- [222] Nadia Thalmann and Daniel Thalmann. 1978. The use of PASCAL as a teaching tool in introductory, intermediate and advanced computer science courses. In *Papers of the SIGCSE/CSA Technical Symposium on Computer Science Education (SIGCSE '78)*. ACM, New York, NY, 277–281. DOI: <https://doi.org/10.1145/990555.990641>
- [223] Anastasios Theodoropoulos and Georgios Lepouras. 2020. Digital game-based learning and computational thinking in P-12 education: a systematic literature review on playing games for learning programming. In *Handbook of Research on Tools for Teaching Computational Thinking in p-12 Education*. Michail Kalogiannakis and Stamatios Papadakis (Eds.), IGI Global (2020), 159–183.
- [224] W. M. To and Billy T. W. Yu. 2023. Rise in higher education researchers and academic publications. *Emerald Open Research* 1, 3 (2023), 1–15.
- [225] Ethel Tshukudu and Quintin Cutts. 2020. Understanding conceptual transfer for students learning new programming languages. In *16th International Computing Education Research Conference (ICER '20)*, 227–237.
- [226] Allen B. Tucker. 1991. Computing curricula 1991. *Communications of the ACM* 34, 6 (1991), 68–84.
- [227] Karthikeyan Umapathy and Albert D. Ritzhaupt. 2017. A meta-analysis of pair-programming in computer programming courses: Implications for educational practice. *ACM Transactions on Computing Education* 17, 4 (2017), 1–13.
- [228] Ian Utting, Allison Elliott Tew, Mike McCracken, Lynda Thomas, Dennis Bouvier, Roger Frye, James Paterson, Michael Caspersen, Yifat Ben-David Kolikant, Juha Sorva, et al. 2013. A fresh look at novice programmers' performance and their teachers' expectations. In *Proceedings of the ITiCSE Working Group Reports Conference on Innovation and Technology in Computer Science Education-Working Group Reports*, 15–32.
- [229] David W. Valentine. 2004. CS educational research: A meta-analysis of SIGCSE technical symposium proceedings. *ACM SIGCSE Bulletin* 36, 1 (2004), 255–259.
- [230] Teemu Valtonen, Sonsoles López-Pernas, Mohammed Saqr, Henriikka Vartiainen, Erkko Tapio Sointu, and Matti Tedre. 2022. The nature and building blocks of educational technology research. *Computers in Human Behavior* 128 (2022), 107123. DOI: <https://doi.org/10.1016/j.chb.2021.107123>
- [231] Andries van Dam, Jens M. Dill, Douglas F. Dixon, and David S. Notkin. 1976. Structured programming in assembly language. *ACM SIGCSE Bulletin* 8, 4 (1976), 53–67. DOI: <https://doi.org/10.1145/382222.382464>
- [232] Anthony Van Raan. 2019. Measuring science: Basic principles and application of advanced bibliometrics. In *Springer Handbook of Science and Technology Indicators*. Wolfgang Glänzel, Henk F. Moed, Ulrich Schmoch, and Mike Thelwall (Eds.), Springer Nature, 237–280.
- [233] Scott Vandenberg and Michael Wollowski. 2000. Introducing computer science using a breadth-first approach and functional programming. In *Proceedings of the 31st SIGCSE Technical Symposium on Computer Science Education*, 180–184.
- [234] Thomas VanDrunen. 2011. The case for teaching functional programming in discrete math. In *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications Companion*, 81–86.
- [235] Henriikka Vartiainen, Tapani Toivonen, Ilkka Jormanainen, Juho Kahila, Matti Tedre, and Teemu Valtonen. 2021. Machine learning for middle schoolers: Learning through data-driven design. *International Journal of Child-Computer Interaction* 29 (2021), 100281. DOI: <https://doi.org/10.1016/j.ijcci.2021.100281>
- [236] Ike Vayansky and Sathish A. P. Kumar. 2020. A review of topic modeling methods. *Information Systems* 94 (2020), 101582.
- [237] Marisa Venter. 2020. Gamification in STEM programming courses: State of the art. In *2020 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, 859–866. DOI: <https://doi.org/10.1109/EDUCON45650.2020.9125395>
- [238] Elena Verdú, Luisa M. Regueras, María J. Verdú, José P. Leal, Juan P. de Castro, and Ricardo Queirós. 2012. A distributed system for learning programming on-line. *Computers and Education* 58, 1 (2012), 1–10. DOI: <https://doi.org/10.1016/j.compedu.2011.08.015>

- [239] Arto Vihavainen, Jonne Airaksinen, and Christopher Watson. 2014. A systematic review of approaches for teaching introductory programming and their influence on success. In *Proceedings of the 10th Annual Conference on International Computing Education Research*, 19–26.
- [240] Xiao-Ming Wang, Gwo-Jen Hwang, Zi-Yun Liang, and Hsiu-Ying Wang. 2017. Enhancing students' computer programming performances, critical thinking awareness and attitudes towards programming: An online peer-assessment attempt. *Journal of Educational Technology & Society* 20, 4 (2017), 58–68.
- [241] David Weintrop and Uri Wilensky. 2017. Comparing block-based and text-based programming in high school computer science classrooms. *ACM Transactions on Computing Education* 18, 1 (October 2017), Article 3, 1–25. DOI: <https://doi.org/10.1145/3089799>
- [242] David Weintrop and Uri Wilensky. 2019. Transitioning from introductory block-based and text-based environments to professional programming languages in high school computer science classrooms. *Computers and Education* 142, C (December 2019), 1–17. DOI: <https://doi.org/10.1016/j.compedu.2019.103646>
- [243] Michel Wermelinger. 2023. Using GitHub Copilot to solve simple programming problems. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE '23)*. ACM, New York, NY, 172–178. DOI: <https://doi.org/10.1145/3545945.3569830>
- [244] Linda Werner, Jill Denner, Shannon Campe, and Damon Chizuru Kawamoto. 2012. The fairy performance assessment: Measuring computational thinking in middle school. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE '12)*. ACM, New York, NY, 215–220. DOI: <https://doi.org/10.1145/2157136.2157200>
- [245] Jeannette M. Wing. 2006. Computational thinking. *Communications of the ACM* 49, 3 (2006), 33–35.
- [246] Jeannette M. Wing. 2008. Five deep questions in computing. *Communications of the ACM* 51, 1 (January 2008), 58–60.
- [247] David Wolber. 2011. App Inventor and real-world motivation. In *Proceedings of the 42nd ACM Technical Symposium on Computer Science Education (SIGCSE '11)*. ACM, New York, NY, 601–606. DOI: <https://doi.org/10.1145/1953163.1953329>
- [248] Benjamin Xie, Dastyni Loksa, Greg L. Nelson, Matthew J. Davidson, Dongsheng Dong, Harrison Kwik, Alex Hui Tan, Leanne Hwa, Min Li, and Amy J. Ko. 2019. A theory of instruction for introductory programming skills. *Computer Science Education* 29, 2–3 (2019), 205–253.
- [249] Alok Kumar Yadav and Solomon Sunday Oyelere. 2021. Contextualized mobile game-based learning application for computing education. *Education and Information Technologies* 26, 3 (2021), 2539–2562. DOI: <https://doi.org/10.1007/s10639-020-10373-3>
- [250] Edward A. Youngs. 1974. Human errors in programming. *International Journal of Man-Machine Studies* 6, 3 (1974), 361–376.

Appendix A

With our existing knowledge of the reputable conferences, journals, and magazines within the CER field, our choice of venues where we included all papers in the initial dataset is as follows:

- *CER-Focused Conferences*:
 - SIGCSE Technical Symposium (SIGCSE)
 - Innovation and Technology in Computer Science Education (ITiCSE)
 - Koli Calling International Conference on Computing Education Research (Koli Calling)
 - International Computing Education Research Conference (ICER)
 - Global Computing Education Conference (CompED)
 - Computer Science Education Research Conference (CSERC)
 - International Conference on Informatics in Schools (ISSEP)
 - Workshop in Primary and Secondary Computing Education (WiPSCE)
 - UK and Ireland Computing Education Research (UKICER)
 - Australasian Computing Education Conference (ACE)
- *CER-Dedicated Journals*:
 - ACM Transactions on Computing Education (TOCE)
 - Computer Science Education (CSE)
- *CER-Dedicated Magazines*:
 - ACM SIGCSE Bulletin
 - ACM Inroads

Additional CER-focused venues that were identified during the data cleaning process:

- INTERNATIONAL CONFERENCE ON COMPUTER SCIENCE AND EDUCATION, ICCSE
- IEEE-CS CONFERENCE ON SOFTWARE ENGINEERING EDUCATION AND TRAINING, CSEE AND T
- INFORMATION SYSTEMS EDUCATORS CONFERENCE, ISECON
- GAMIFICATION-BASED E-LEARNING STRATEGIES FOR COMPUTER PROGRAMMING EDUCATION
- JOURNAL OF INFORMATION SYSTEMS EDUCATION
- JOURNAL OF INFORMATION TECHNOLOGY EDUCATION: INNOVATIONS IN PRACTICE
- JOURNAL OF INFORMATION TECHNOLOGY EDUCATION: RESEARCH
- OLYMPIADS IN INFORMATICS
- INTERNATIONAL CONFERENCE ON COMPUTATIONAL THINKING EDUCATION
- WESTERN CANADIAN CONFERENCE ON COMPUTING EDUCATION, WCCCE
- CONFERENCE ON INFORMATION TECHNOLOGY EDUCATION, SIGITE
- SOFTWARE ENGINEERING EDUCATION CONFERENCE
- SOFTWARE ENGINEERING: EFFECTIVE TEACHING AND LEARNING APPROACHES AND PRACTICES
- WORKSHOP ON TEACHING NATURAL LANGUAGE PROCESSING
- THE CAMBRIDGE HANDBOOK OF COMPUTING EDUCATION RESEARCH

Received 14 August 2024; revised 25 August 2025; accepted 28 August 2025